



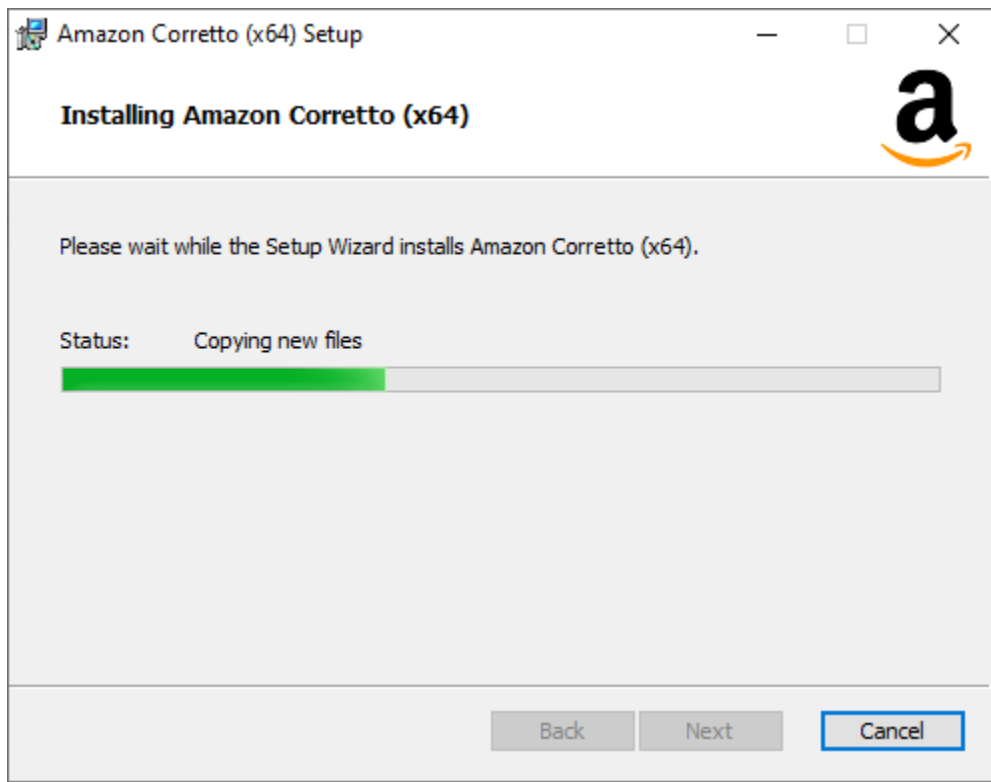
INSTALLING WS2O MICRO INTEGRATOR

1. Useful links :
 - URL of WSO2 Micro Integrator site : <https://wso2.com/integrator/micro-integrator/>
 - URL for WS02 Micro Integrator documentation : <https://mi.docs.wso2.com/en/latest/>
2. Pre-Req for installation: Install JDK

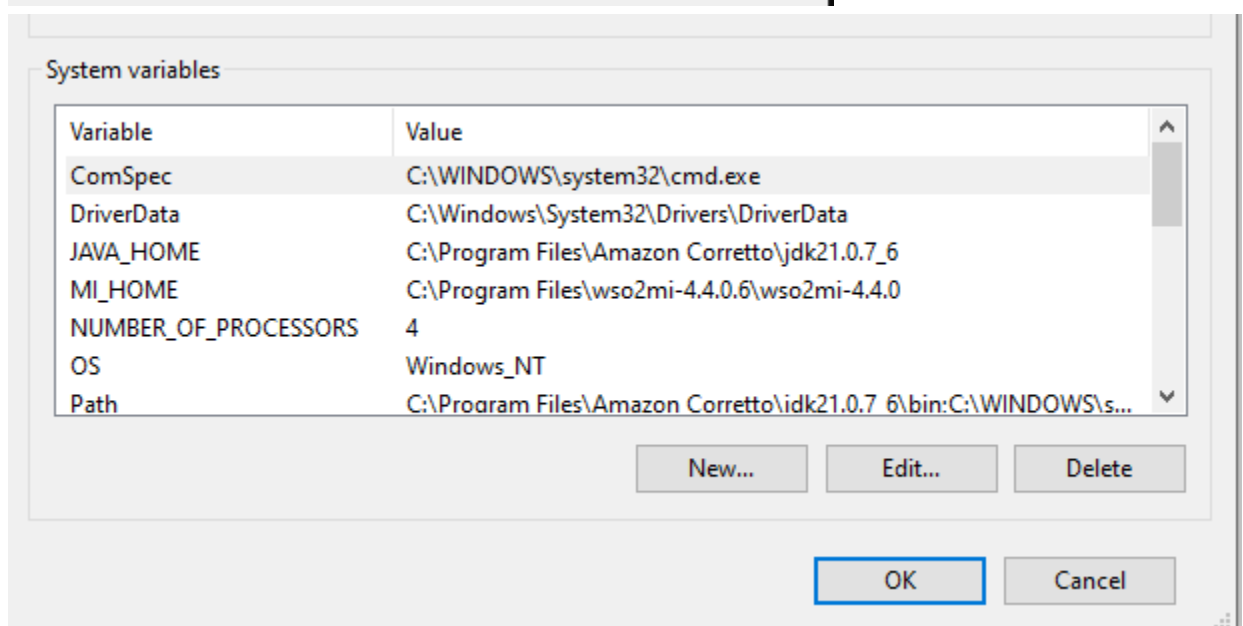
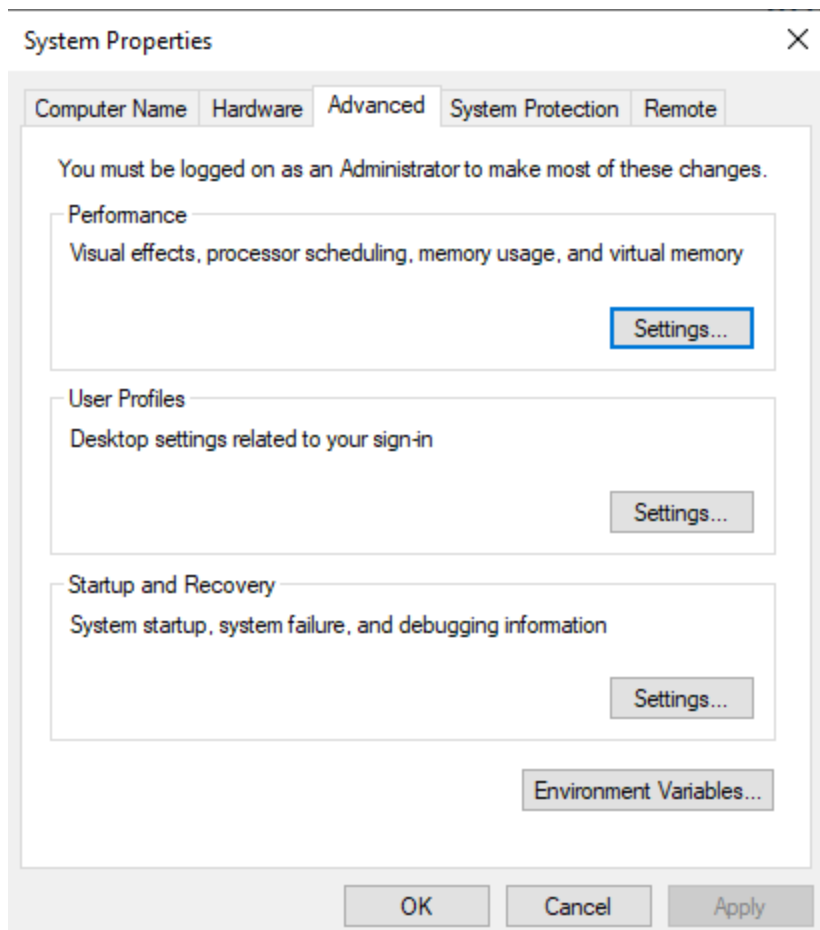
JDK	Versions
CorrettoJDK	11, 17, 21
AdoptOpenJDK	11, 17, 21
OpenJDK	11, 17, 21
Oracle JDK	11, 17, 21

- Download JDK from Amazon JDK (corretto) : <https://docs.aws.amazon.com/corretto/latest/corretto-21-ug/downloads-list.html>
- Windows x64 : <https://corretto.aws/downloads/latest/amazon-corretto-21-x64-windows-jdk.msi>

- Run the installer :



3. Download WSO2 Micro Integrator Installer :
<https://wso2.com/integrator/micro-integrator/#>
4. The download link will be sent to email, download all, wso2 micro-integrator, ide and cli.
5. Extract WSO2 Micro Integrator to program files.
6. Setup JAVA_HOME & MI_HOME in environment variables



STARTING/STOPPING WSO2

1. Starting the MI Server

```
C:\Users\ariad>cd %MI_HOME%
```

```
C:\Program Files\wso2mi-4.4.0.6\wso2mi-4.4.0>cd bin
```

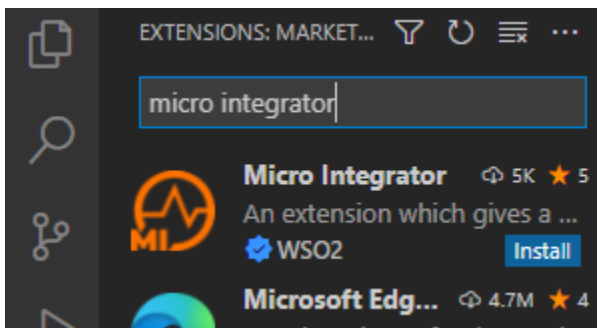
```
C:\Program Files\wso2mi-4.4.0.6\wso2mi-4.4.0\bin>micro-integrator.bat  
[2025-05-02 00:00:20,979] INFO {org.wso2.config.mapper.ConfigParser} - Overriding files in configuration directory C:\P  
ROGRA~1\WSO2MI~1.6\WSO2MI~1.0\bin\..  
[2025-05-02 00:00:21,303] INFO {org.wso2.config.mapper.ConfigParser} - Applying configurations with deployment configur  
ations
```

2. Stopping MI Server

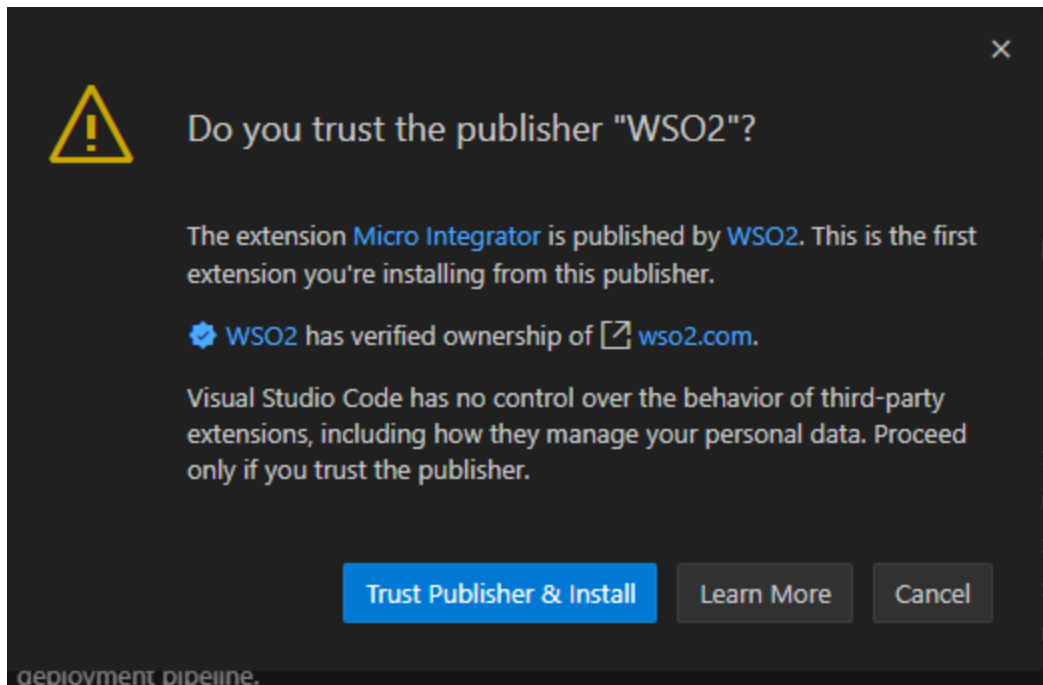
```
C:\Program Files\wso2mi-4.4.0.6\wso2mi-4.4.0\bin>micro-integrator.bat --stop  
Stopping the Micro Integrator Server  
SUCCESS: The process with PID 18704 has been terminated.  
C:\Program Files\wso2mi-4.4.0.6\wso2mi-4.4.0\bin>
```

INSTALLING VS CODE

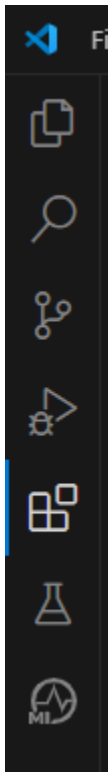
1. Download installer : <https://code.visualstudio.com/download>
2. Install VS Code studio
3. Run VS Code Studio
4. Click View -> Extension
5. Search 'micro integrator'



6. Install the extension



Trust publisher & install

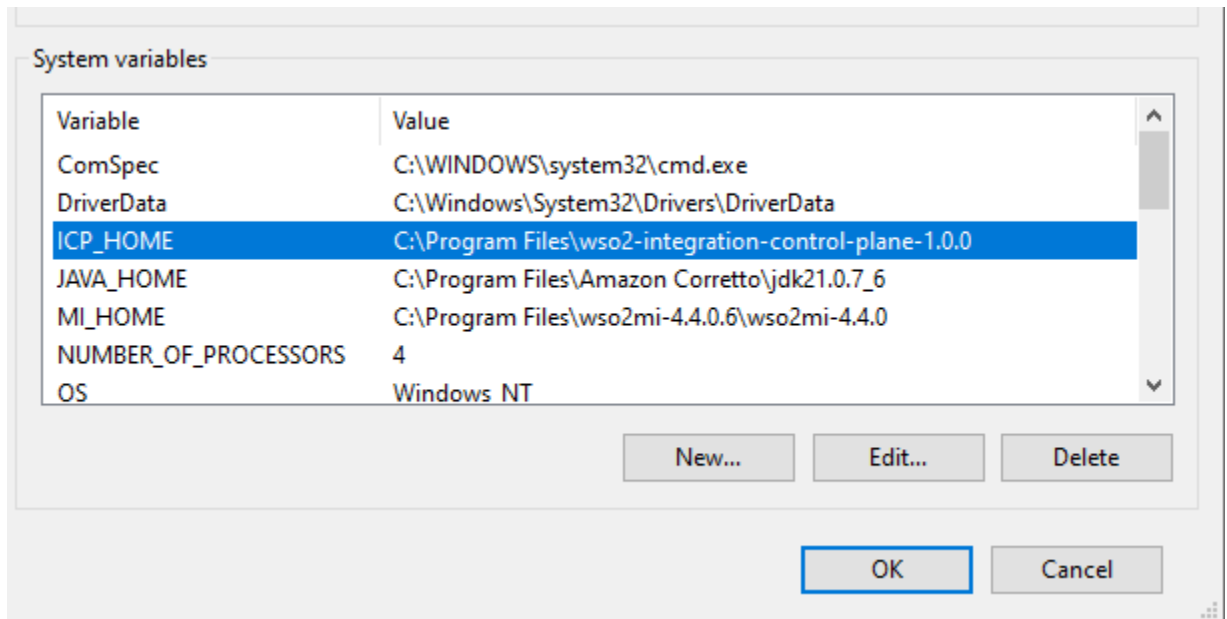


When installation is finished, MI Icon will appear in vs code studio.

INSTALLING WSO2 INTEGRATION CONTROL PLANE

URL for download : <https://wso2.com/integrator/integration-control-plane/>

1. Extract the zip file into program files
2. Setup ICP_HOME as the new environment variables



3. Edit the deployment.toml in MI_HOME/conf

Add the following code

[dashboard_config]

dashboard_url = "https://localhost:9743/dashboard/api/"

heartbeat_interval = 5

group_id = "mi_dev"

node_id = "dev_node_2"

Edit the deployment.toml in ICP_HOME/conf

Add the following code

[sso]

enable = true

client_id = "8e4uDF4ewc2aEa"

base_url = "https://localhost:9443"

jwt_issuer = "https://localhost:9443/oauth2/token"

resource_server_URLs = ["https://localhost:9743"]

sign_in_redirect_URL = <https://localhost:9743/sso>

STARTING THE WSO2 CONTROL PLANE

1. Go to ICP_HOME/bin
cd %ICP_HOME%
cd bin
2. Run the wso2 control plane

```
C:\Program Files\ws2-integration-control-plane-1.0.0\bin>dashboard.bat
[2025-05-02 16:38:41,008] INFO {ConfigParser} - Initializing configurations with deployment configurations
[2025-05-02 16:38:44,668] INFO {DashboardServer} - WS02 Integration Control Plane started.
[2025-05-02 16:38:44,662] INFO {DashboardServer} - Login to Integration Control Plane Dashboard : 'https://DESKTOP-OE0H
NAB3:9743/login'
[2025-05-02 16:38:47,419] INFO {HeartBeatDelegate} - New node dev_node_2 in group : mi_dev is registered. Inserting hea
rtbeat information
[2025-05-02 16:38:48,458] INFO {InMemoryDataManager} - Inserting heartbeat details of node dev_node_2 in group mi_dev
[2025-05-02 16:38:48,461] INFO {MiArtifactsManager} - Fetching server details from node dev_node_2 in group mi_dev
[2025-05-02 16:38:48,496] INFO {InMemoryDataManager} - Adding serverInfo of node dev_node_2 in group mi_dev
SUCCESS: THE PROCESS WITH PID 1028 HAS BEEN TERMINATED.
```

ACCESSING THE INTEGRATOR CONTROL PLANE

1. Type <https://localhost:9743/login>
2. Login into the dashboard

← → ↻ Not secure https://localhost:9743/login


INTEGRATION CONTROL PLANE

Sign in

Username *

admin

Password *

.....|

☐ Remember me

Browser cookies are used to track your session and provide a better experience. You can refer our [Cookie Policy](#) for more details.

By signing in, you agree to our [Privacy Policy](#).

3. Login with default username and password : admin/admin

STOPPING THE INTEGRATOR CONTROL PLANE

1. Go to ICP_HOME/bin
2. Running the dashboard.bat --stop

```
C:\Program Files\wso2-integration-control-plane-1.0.0\bin>dashboard.bat --stop
Stopping the Integration Control Plane
SUCCESS: The process with PID 11804 has been terminated.
```

CREATING HELLO WORLD SERVICE

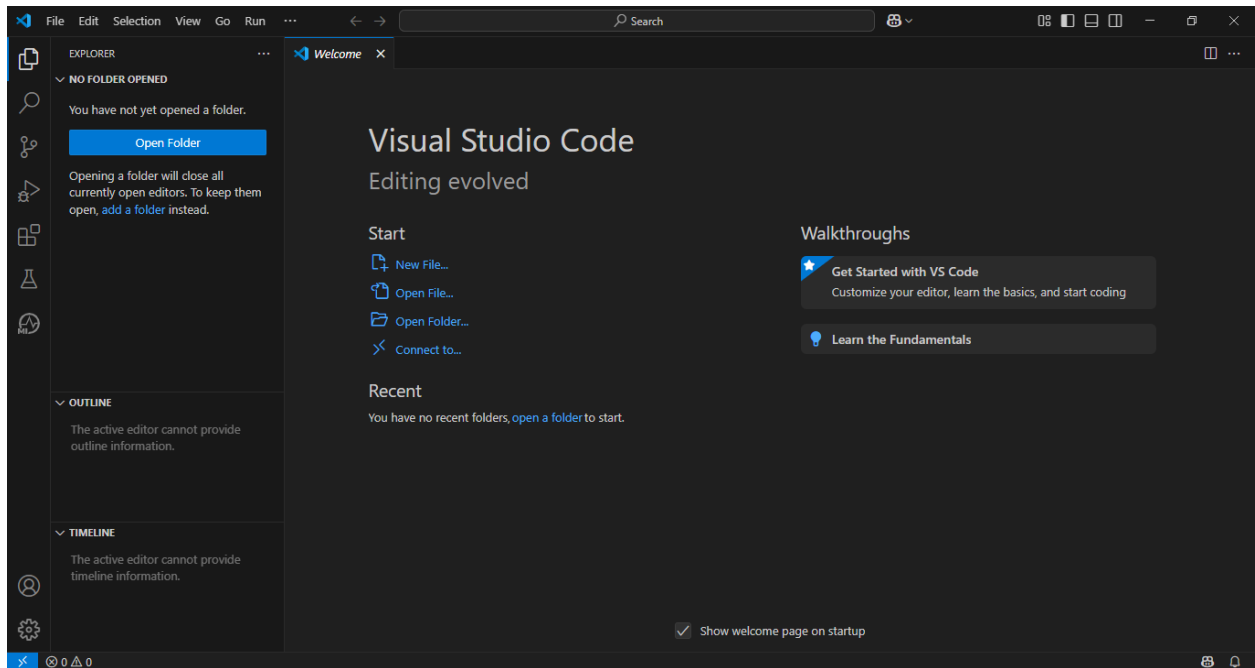
Default installation path for micro integrator runtime in visual studio code extension :

URL for Hello World backend

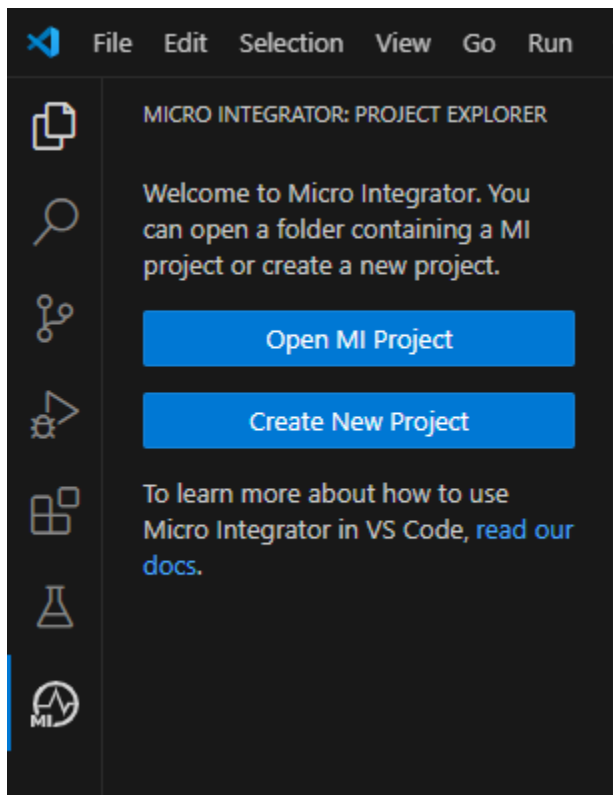
URL	https://apis.wso2.com/zvdz/mi-qsg/v1.0
HTTP Method	GET

C:\Users\ariad\.wso2-mi\micro-integrator\wso2mi-4.4.0

1. Launch visual Studio Code



2. Create new project



Click create new project

Fill in project name and micro integrator runtime version, also specify the project directory

 Project Creation Form ×

Create New Project ×

Project Name*

Micro Integrator runtime version*

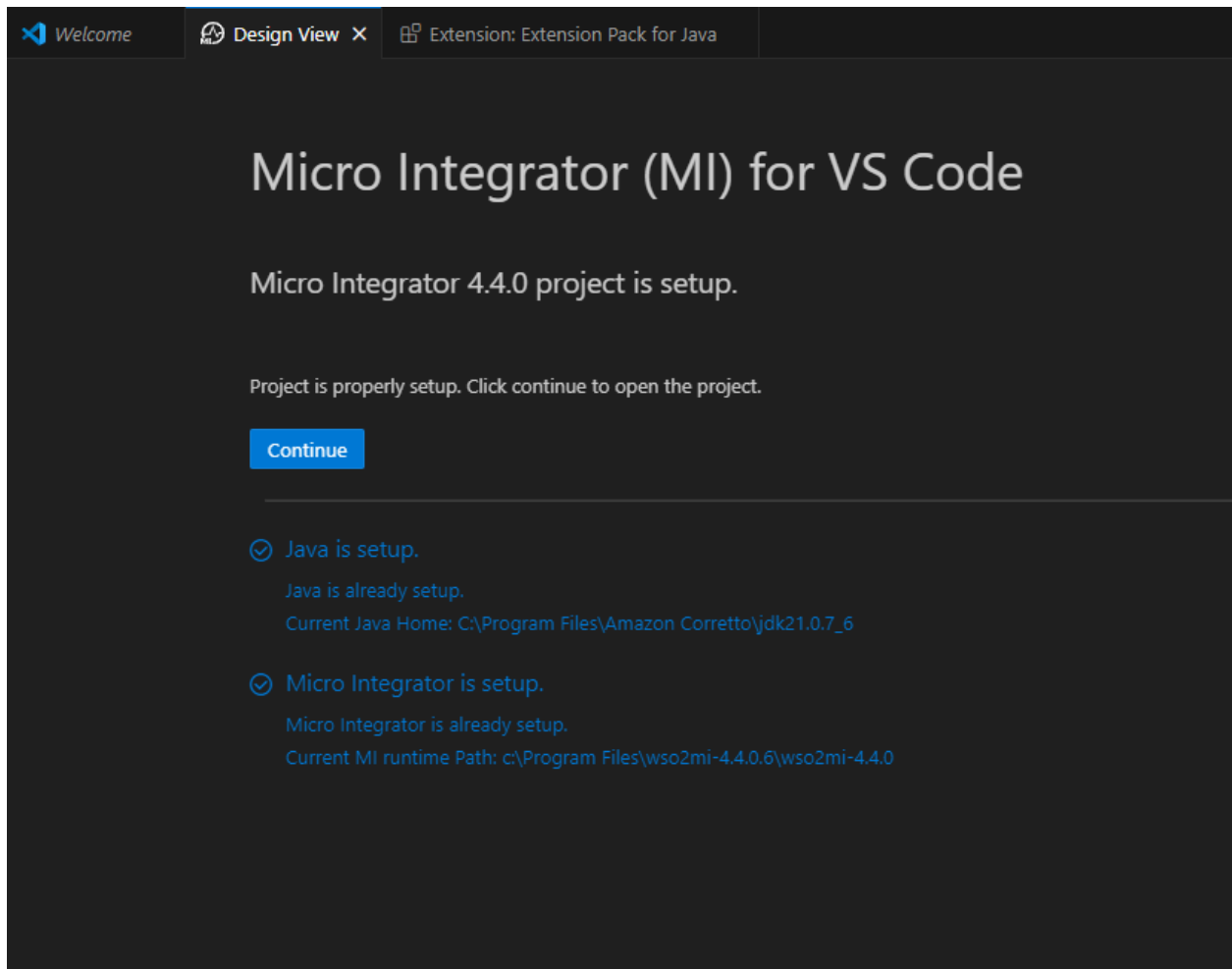
Project Directory*

C:\Users\ariad\wso2mi\Projects

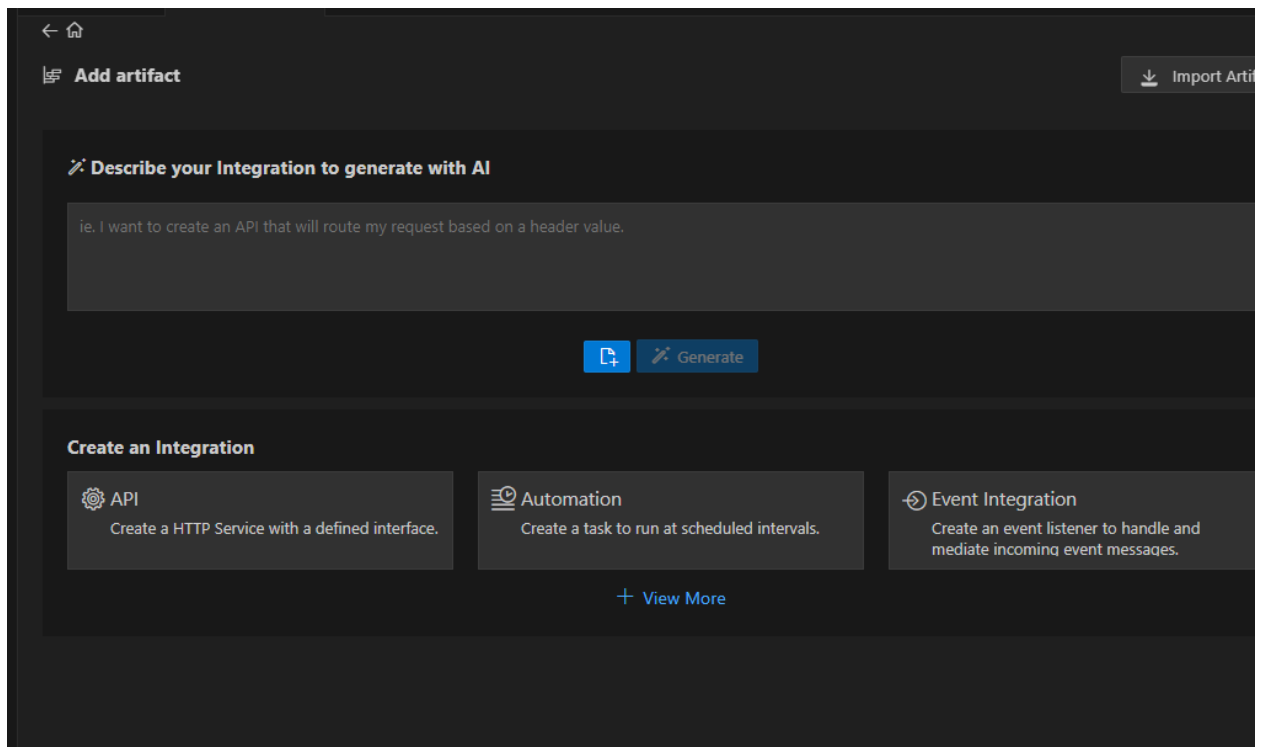
Advanced Options ⋮

If the necessary Micro Integrator runtime and tools are not available, you will be prompted to download them after project creation.

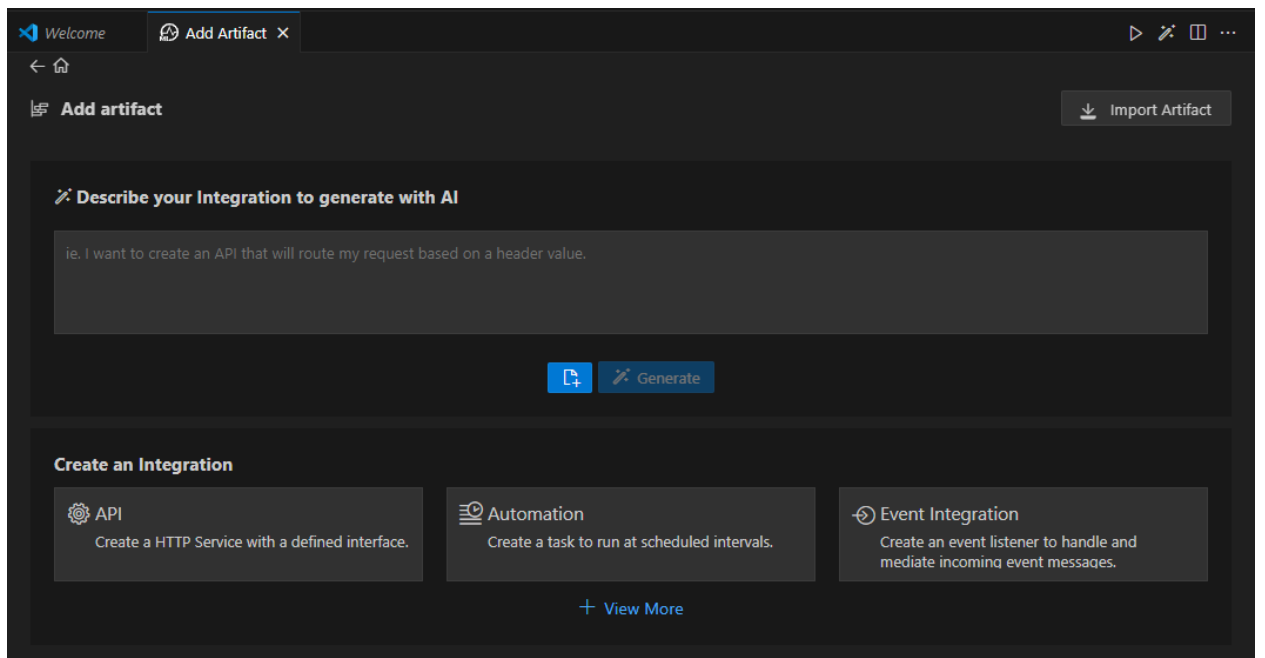
Click Create



Make sure that Java and Micro Integrator is already setup and point to the correct installation path.



Create the API by clicking API



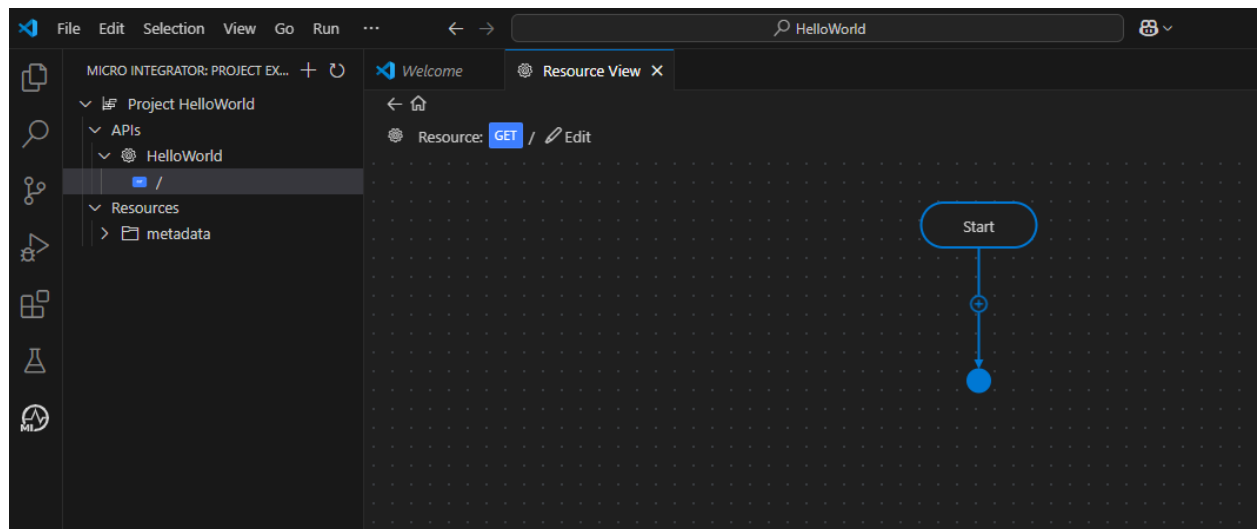
Fill the API name , click create

The screenshot shows the 'Create API' dialog box in the Micro Integrator IDE. The dialog has a title bar with 'Welcome' and 'API Form' tabs. The main content area is titled 'Create API' and contains the following fields and options:

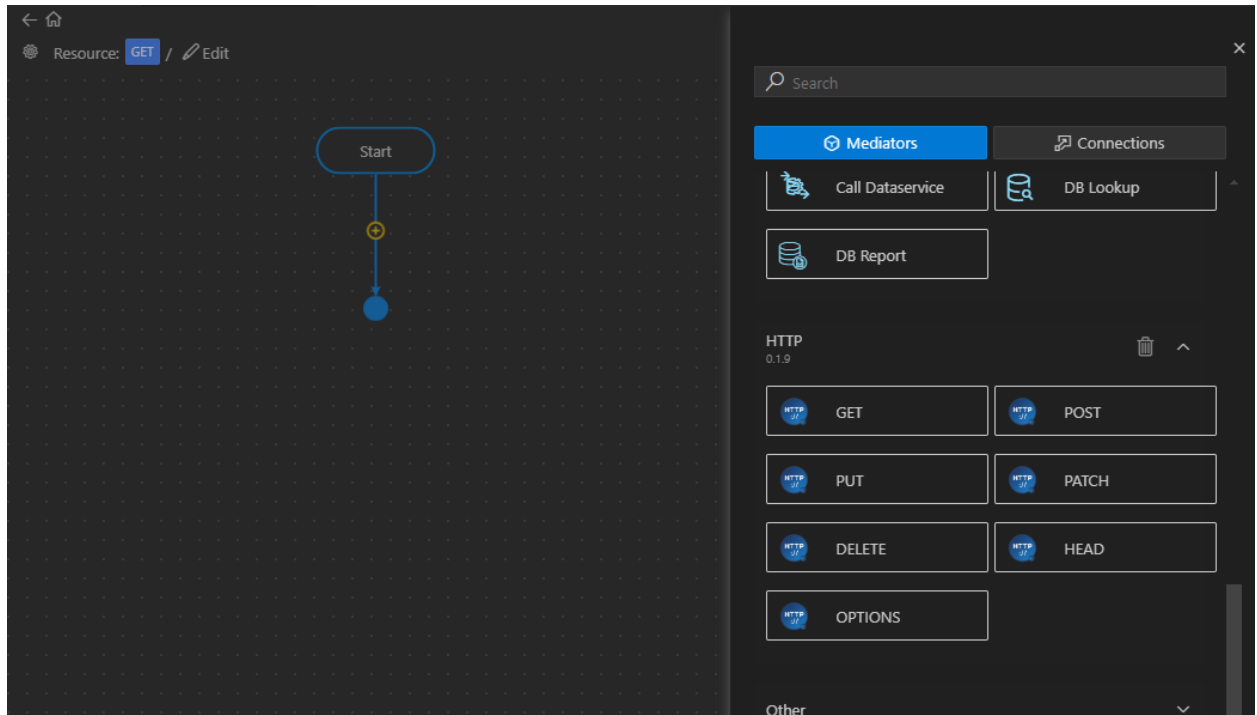
- Name***: A text input field containing 'HelloWorld'.
- Context***: A text input field containing '/helloworld'.
- Version Type**: A dropdown menu currently set to 'None'.
- Generate API From**: A group of three radio buttons:
 - ☒ None
 - ☐ From OpenAPI definition
 - ☐ From WSDL file

At the bottom right of the dialog are two buttons: 'Cancel' and 'Create'.

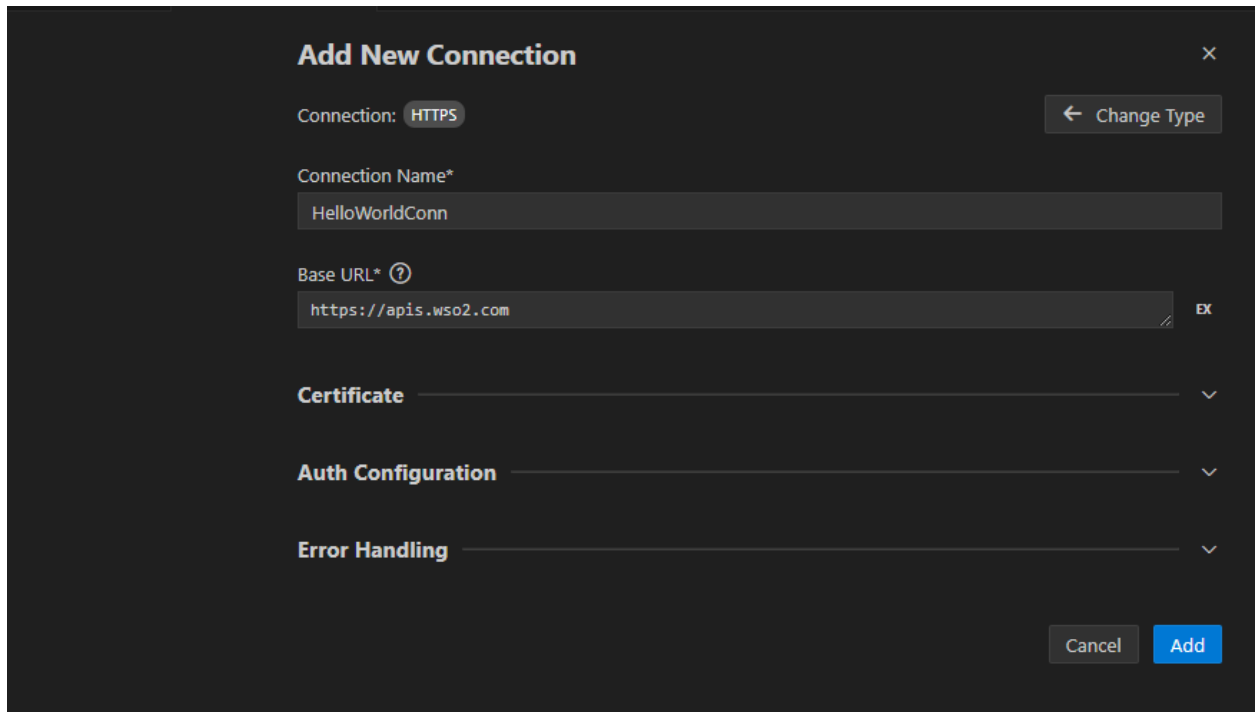
From resource view click +



Under mediators, click HTTP and choose GET



Click add new connection, Select HTTPS
Fill in the connection and click create



Fill in the relative path, then click Add

HTTP

://

Add Get

⏮

⏭

⌕

/zvdz/mi-qsg/v1.0

No query parameters available

+ Add Query Parameter

Headers

Key value pairs fo headers

No data available

+ Add Parameter

Properties

Output

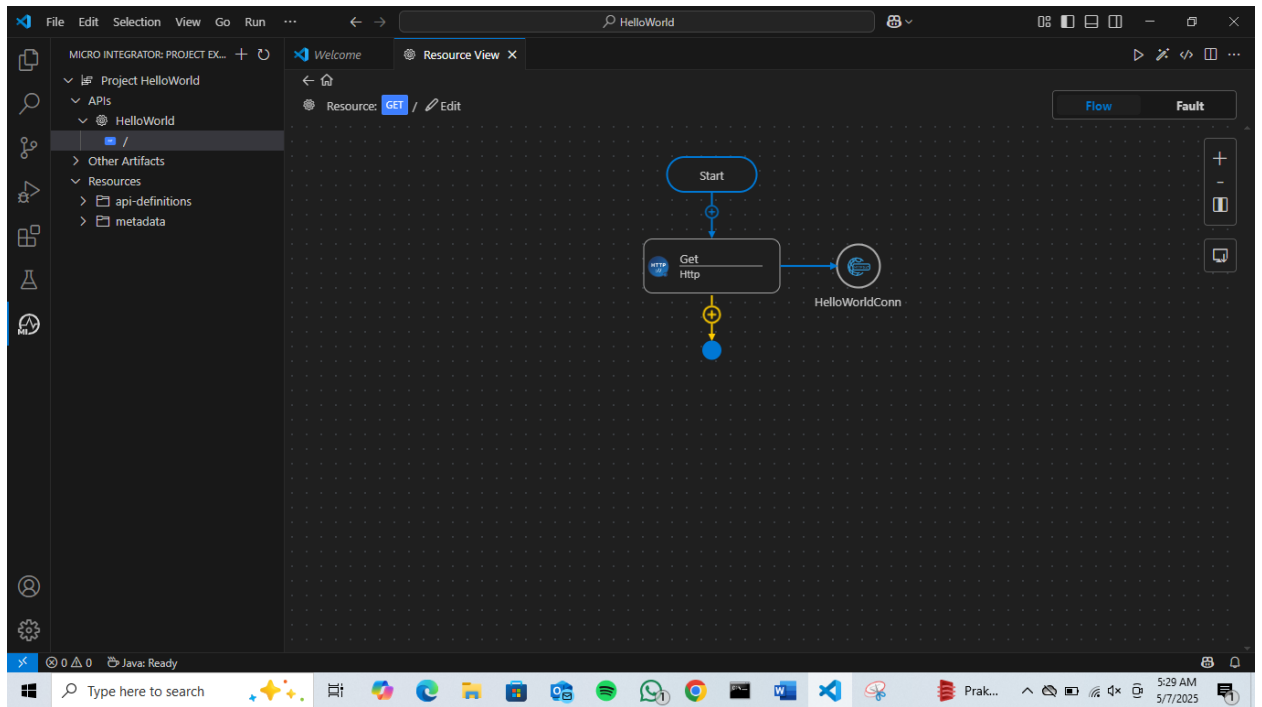
Output Variable Name* ?

http_get_1

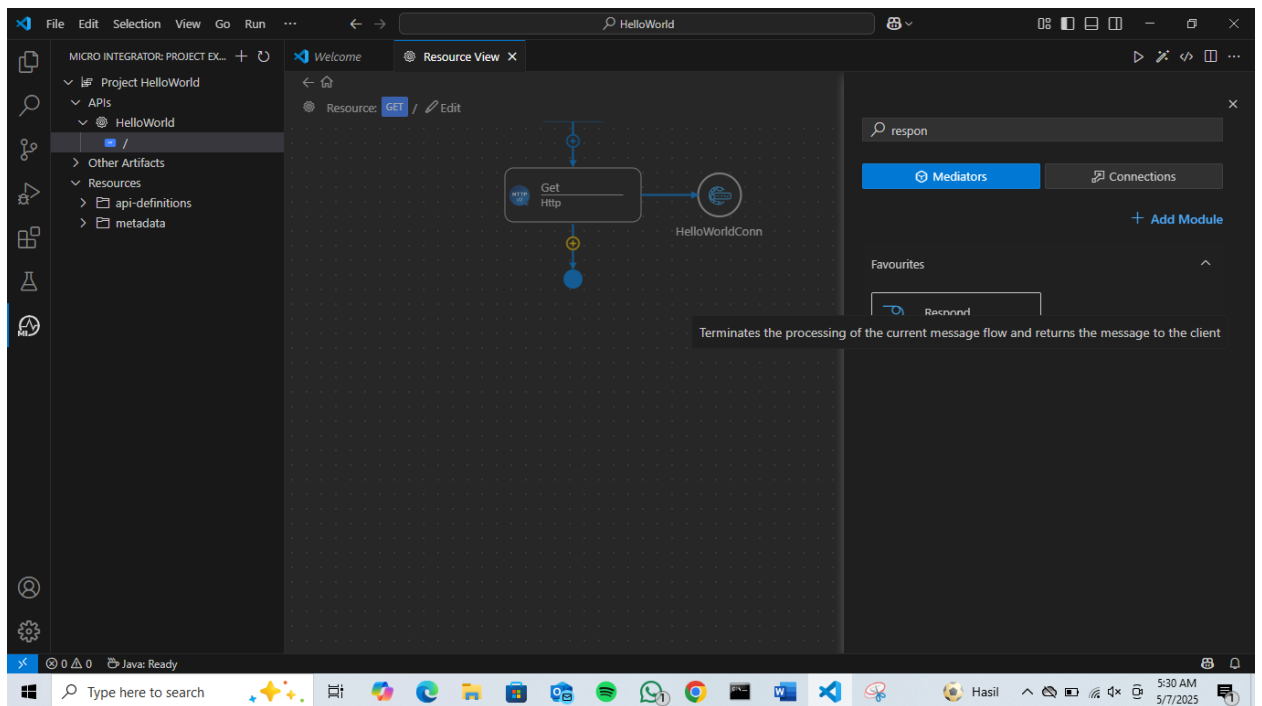
☒ Overwrite Message Body ?

Cancel

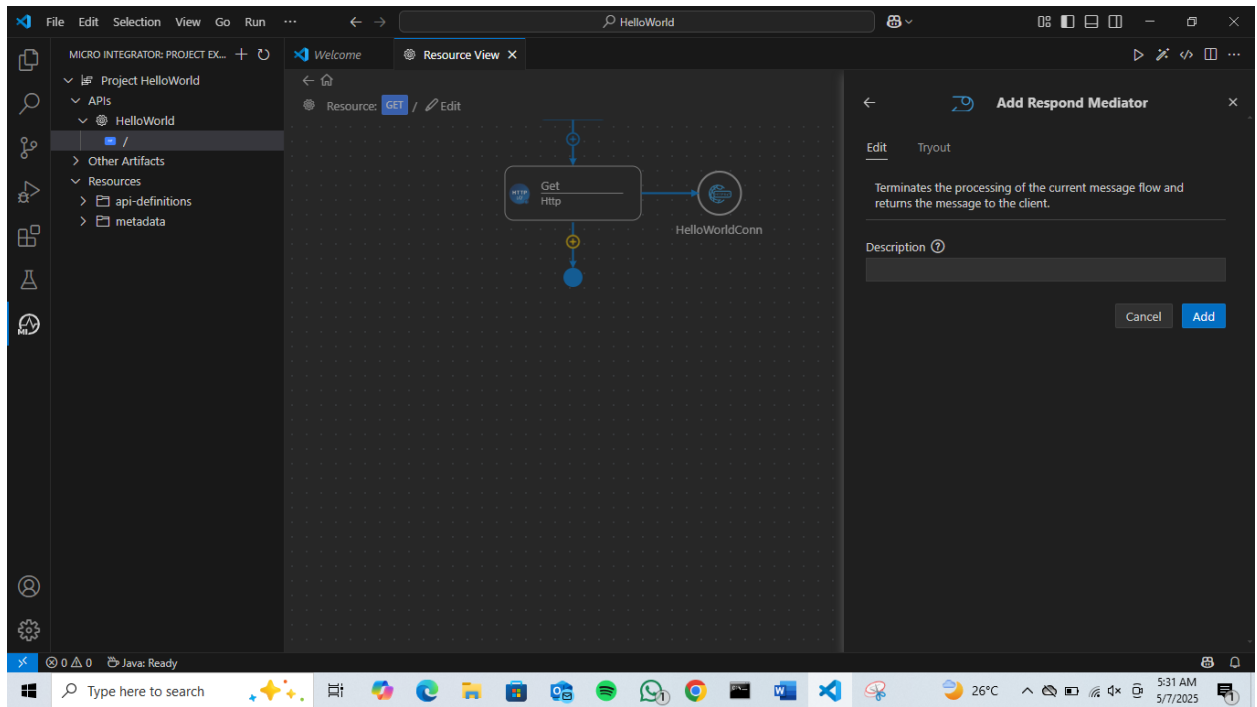
Add



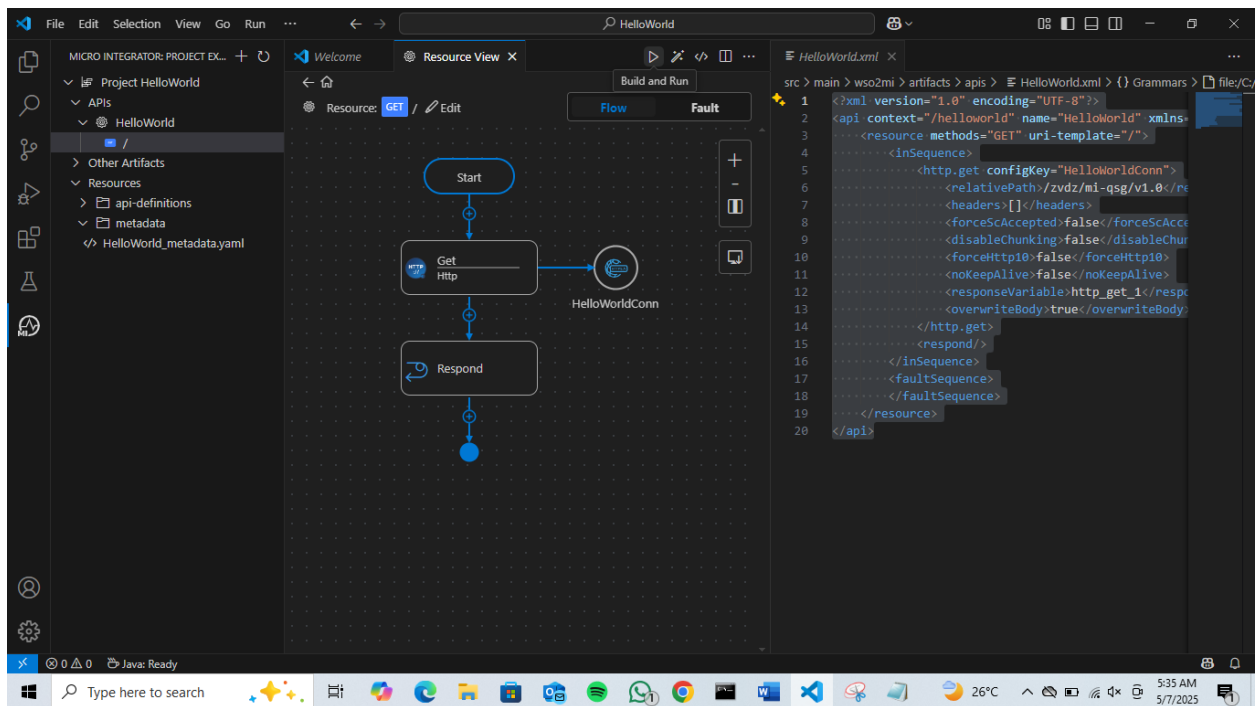
Click +



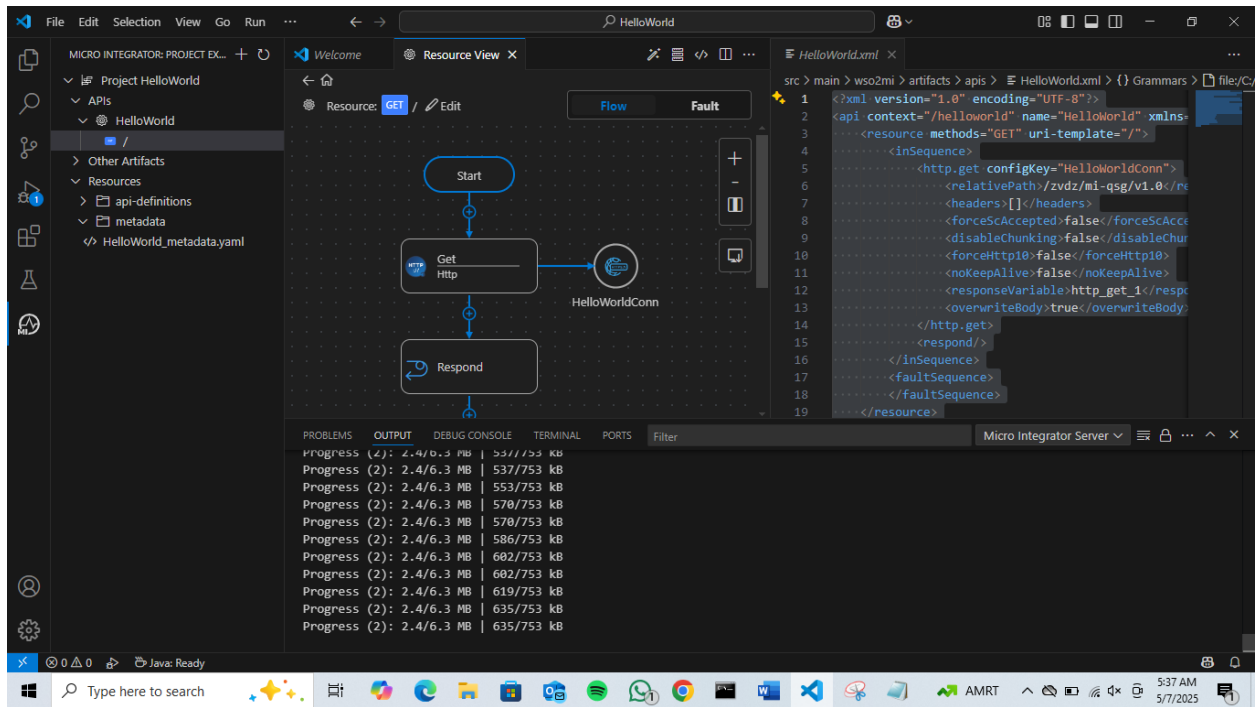
Add respond



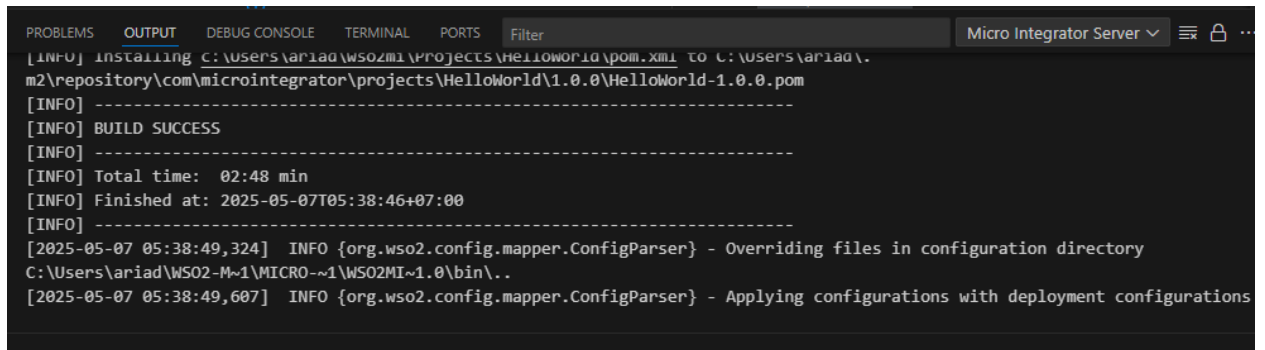
Click Add



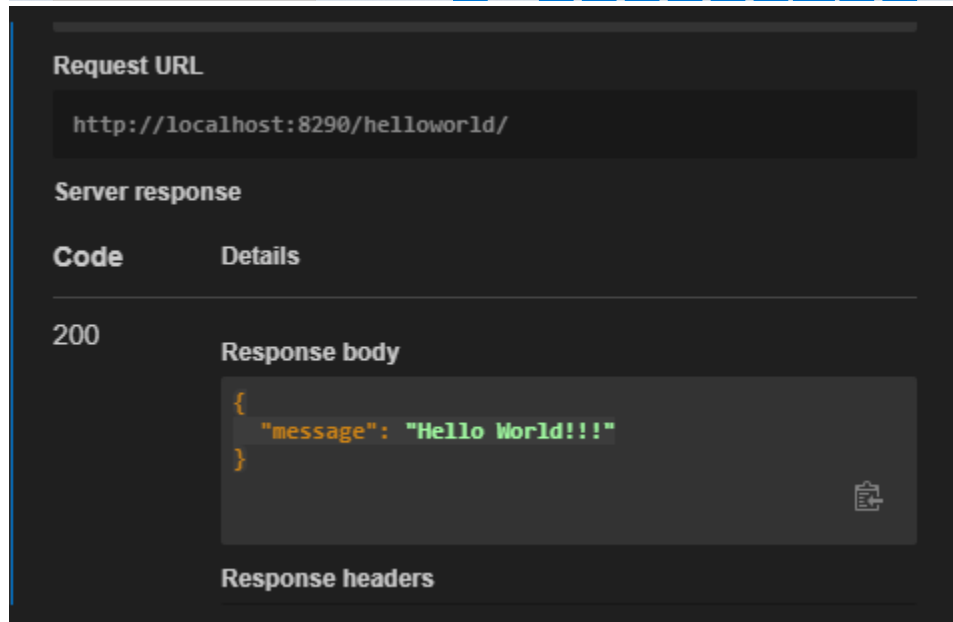
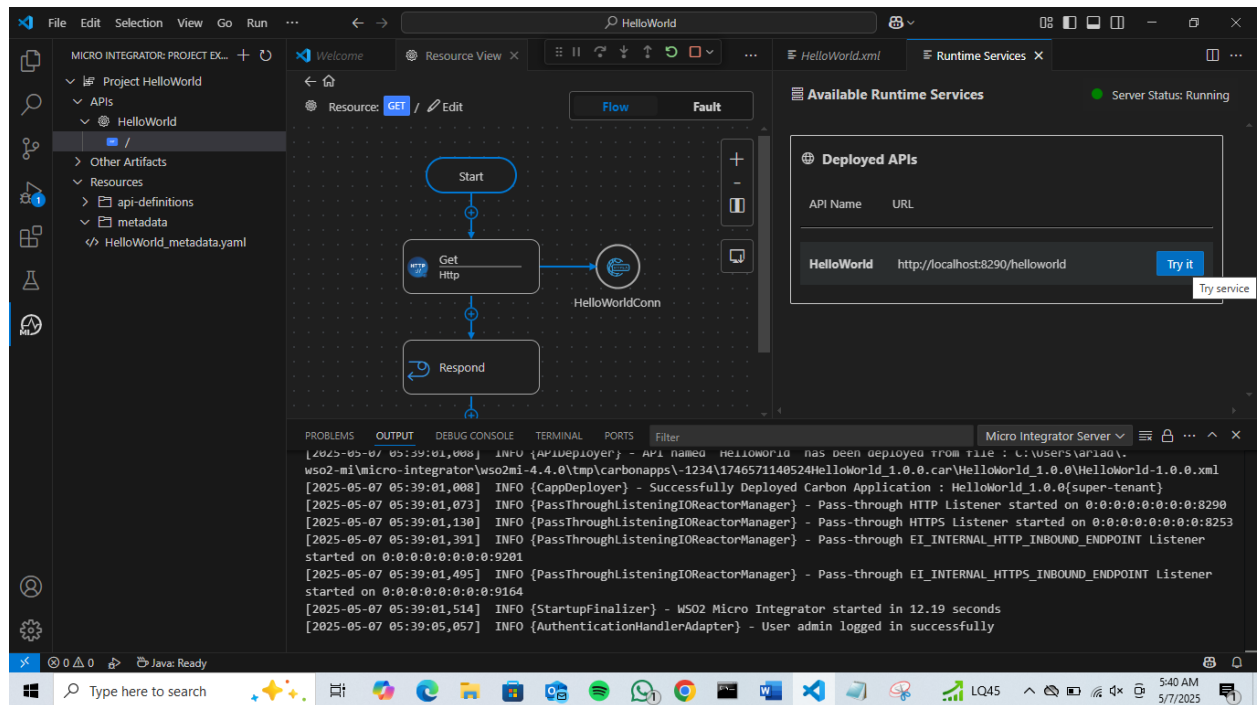
Click build and Run



Wait for the build process to complete

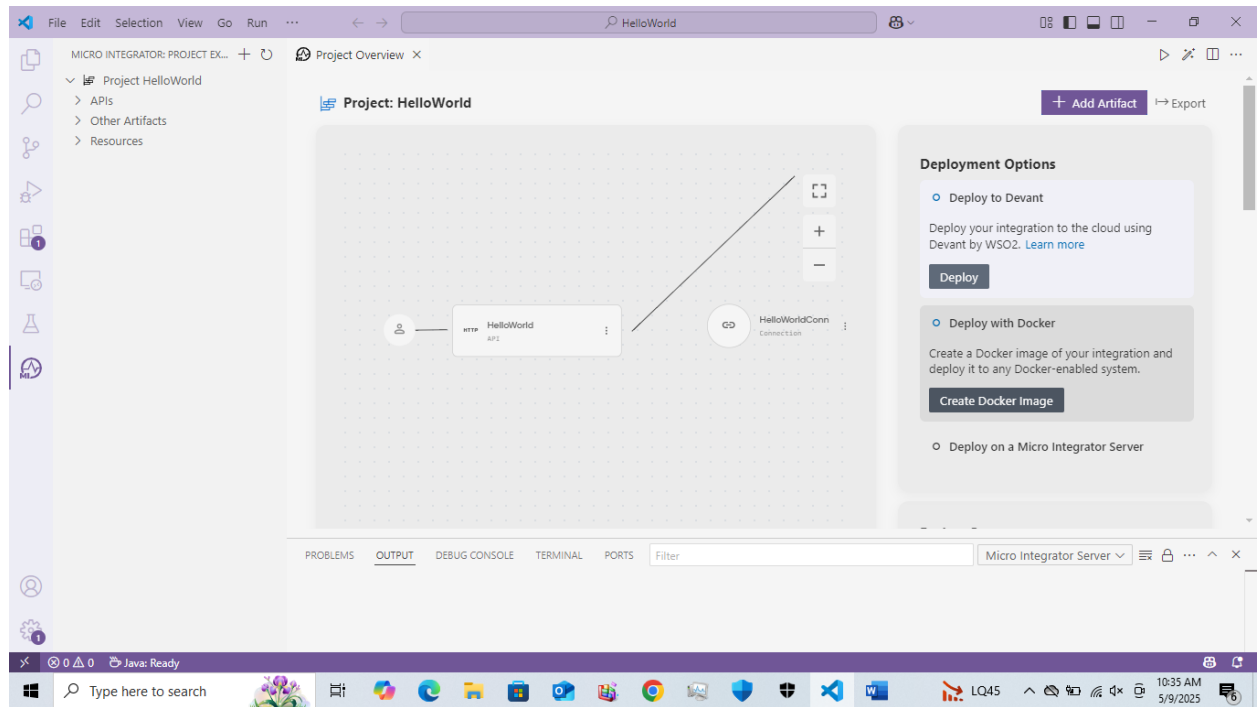


Test the service

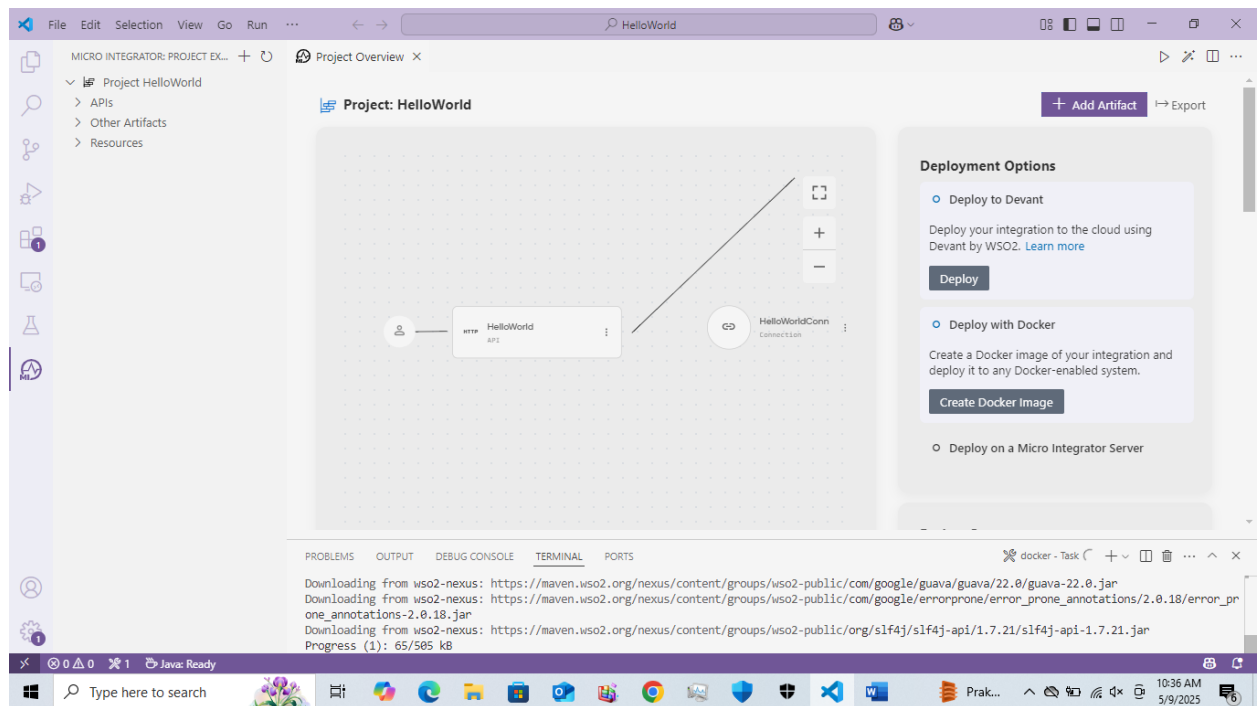


Creating a docker image

Go to Project Overview, then click on create docker image

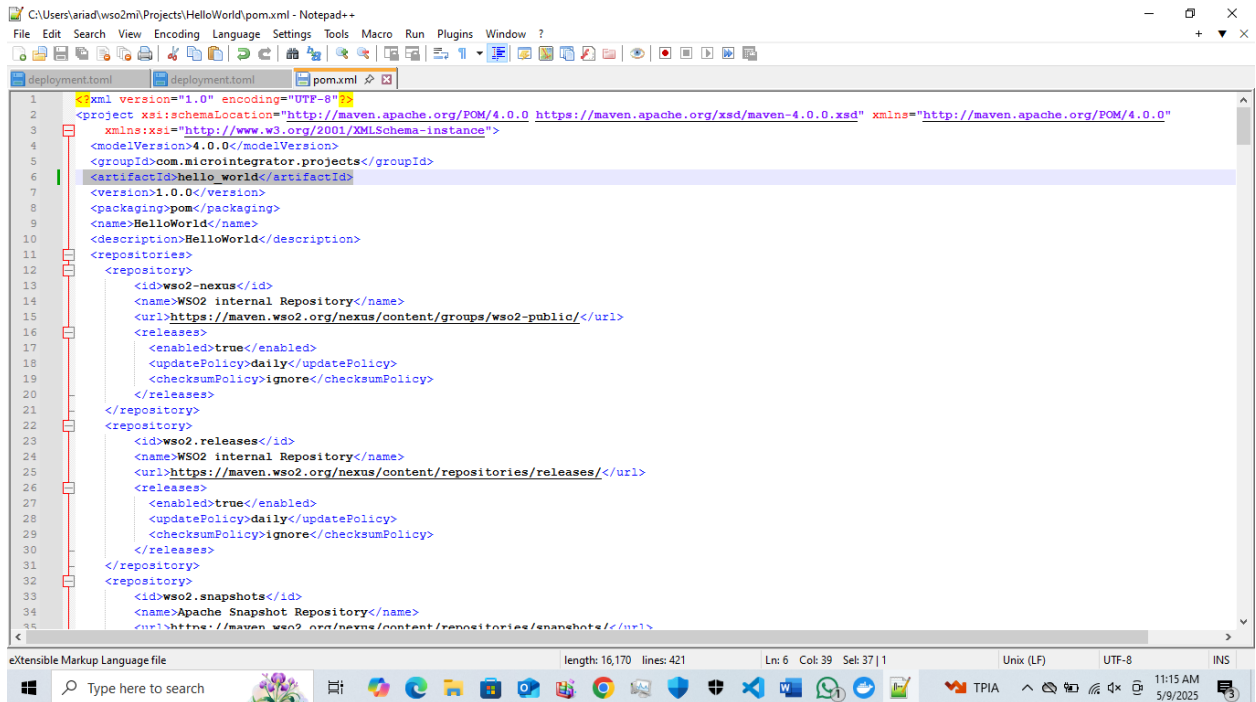


Wait until the process is completed.



- Note:
 - Before building the image make sure the docker desktop is running. If you forgot to run docker desktop, run the docker desktop the re-build the image.

- Make sure the image name conforms to the docker desktop conventions. To fix the image name issue open pom.xml and modify artifactId



A screenshot of the Notepad++ text editor showing a Maven pom.xml file. The file is titled 'pom.xml' and is located at 'C:\Users\ariad\wso2m\Projects\HelloWorld\pom.xml'. The XML content is as follows:

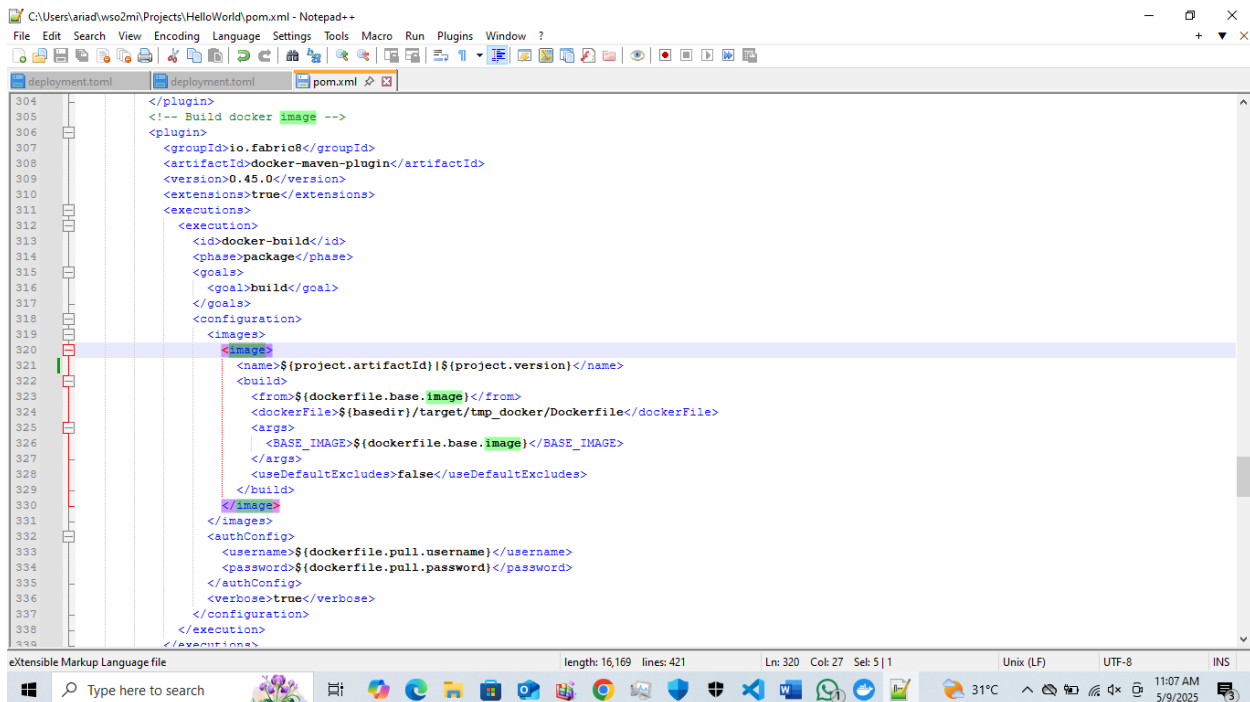
```

1  <?xml version="1.0" encoding="UTF-8"?>
2  <project xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd" xmlns="http://maven.apache.org/POM/4.0.0"
3  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
4  <modelVersion>4.0.0</modelVersion>
5  <groupId>com.microintegrator.projects</groupId>
6  <artifactId>hello_world</artifactId>
7  <version>1.0.0</version>
8  <packaging>pom</packaging>
9  <name>HelloWorld</name>
10 <description>HelloWorld</description>
11 <repositories>
12 <repository>
13 <id>wso2-nexus</id>
14 <name>WSO2 internal Repository</name>
15 <url>https://maven.wso2.org/nexus/content/groups/wso2-public/</url>
16 <releases>
17 <enabled>true</enabled>
18 <updatePolicy>daily</updatePolicy>
19 <checksumPolicy>ignore</checksumPolicy>
20 </releases>
21 </repository>
22 <repository>
23 <id>wso2.releases</id>
24 <name>WSO2 internal Repository</name>
25 <url>https://maven.wso2.org/nexus/content/repositories/releases/</url>
26 <releases>
27 <enabled>true</enabled>
28 <updatePolicy>daily</updatePolicy>
29 <checksumPolicy>ignore</checksumPolicy>
30 </releases>
31 </repository>
32 <repository>
33 <id>wso2.snapshots</id>
34 <name>Apache Snapshot Repository</name>
35 <url>https://maven.wso2.org/nexus/content/repositories/snapshots/</url>

```

The line containing `<artifactId>hello_world</artifactId>` is highlighted in blue. The status bar at the bottom shows 'length: 16,170 lines: 421'.

- If the naming still issue try to edit the <image> part and change the regex pattern.



A screenshot of the Notepad++ text editor showing a Maven pom.xml file. The file is titled 'pom.xml' and is located at 'C:\Users\ariad\wso2m\Projects\HelloWorld\pom.xml'. The XML content is as follows:

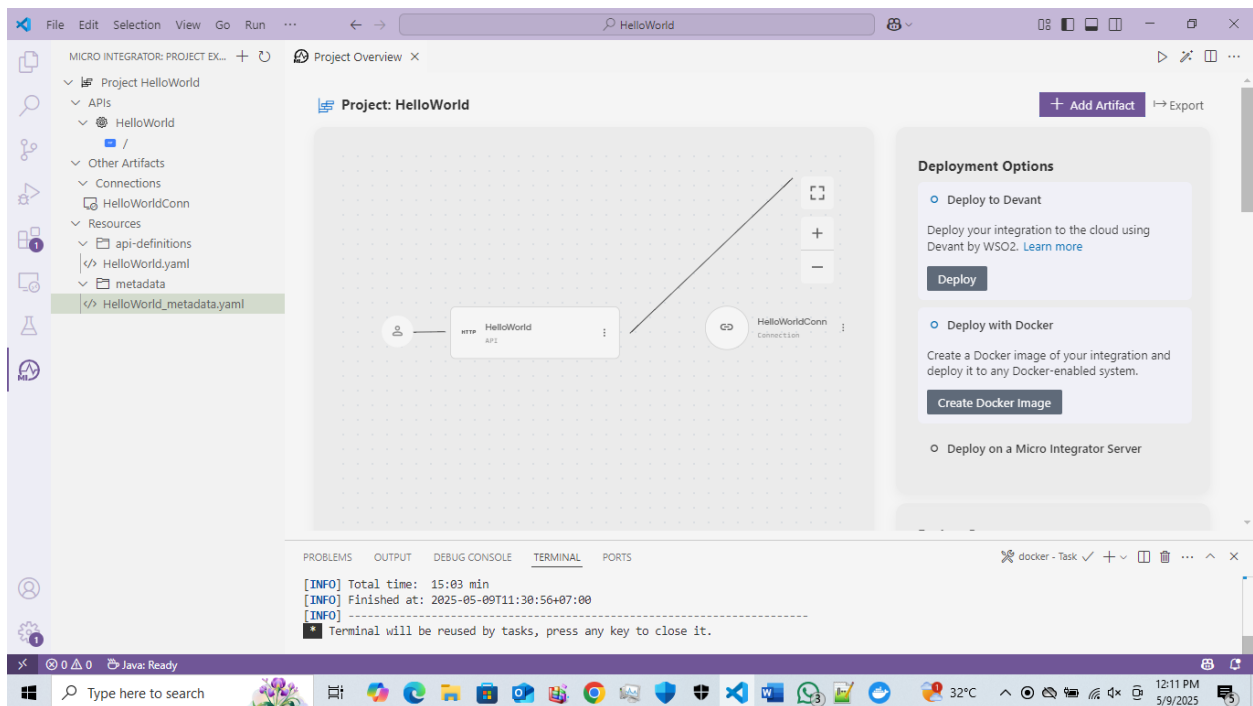
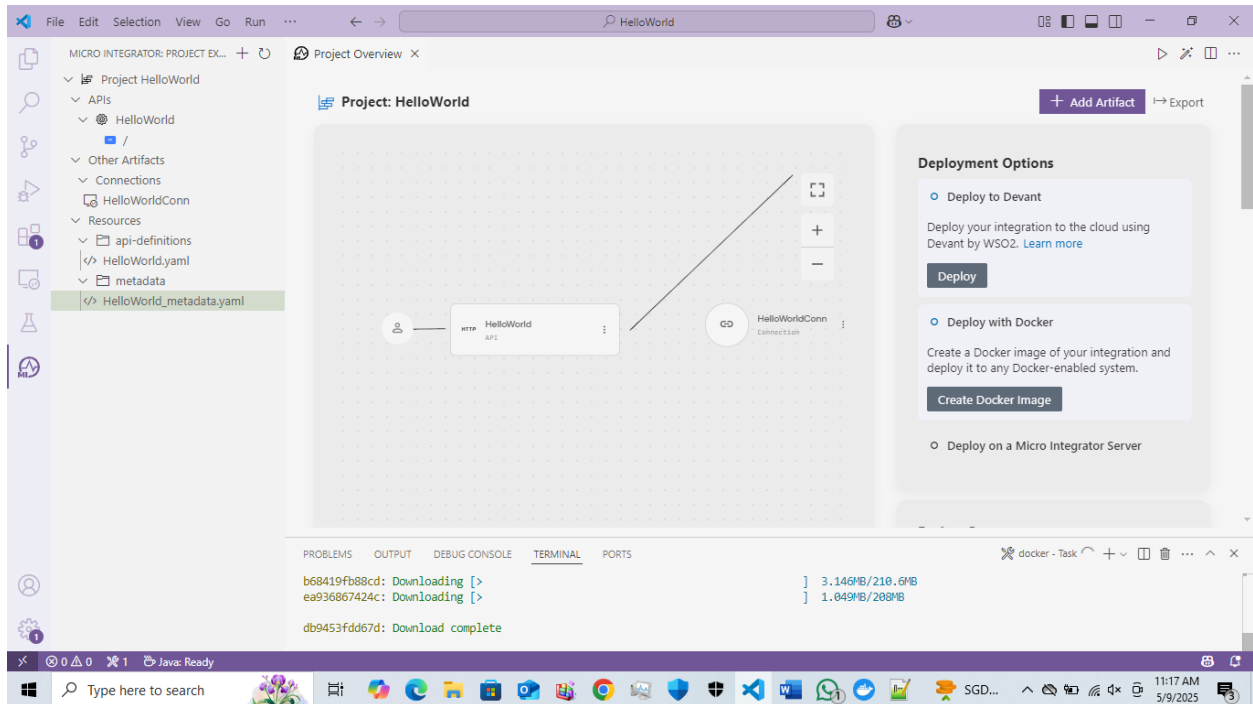
```

304 </plugin>
305 <!-- Build docker image -->
306 <plugin>
307 <groupId>io.fabric8</groupId>
308 <artifactId>docker-maven-plugin</artifactId>
309 <version>0.45.0</version>
310 <extensions>true</extensions>
311 <executions>
312 <execution>
313 <id>docker-build</id>
314 <phase>package</phase>
315 <goals>
316 <goal>build</goal>
317 </goals>
318 <configuration>
319 <images>
320 <image>
321 <name>${project.artifactId}|${project.version}</name>
322 <build>
323 <from>${dockerfile.base_image}</from>
324 <dockerFile>${basedir}/target/tmp_docker/Dockerfile</dockerFile>
325 <args>
326 <BASE_IMAGE>${dockerfile.base_image}</BASE_IMAGE>
327 </args>
328 <useDefaultExcludes>false</useDefaultExcludes>
329 </build>
330 </image>
331 </images>
332 <authConfig>
333 <username>${dockerfile.pull.username}</username>
334 <password>${dockerfile.pull.password}</password>
335 </authConfig>
336 <verbose>true</verbose>
337 </configuration>
338 </execution>
339 </executions>

```

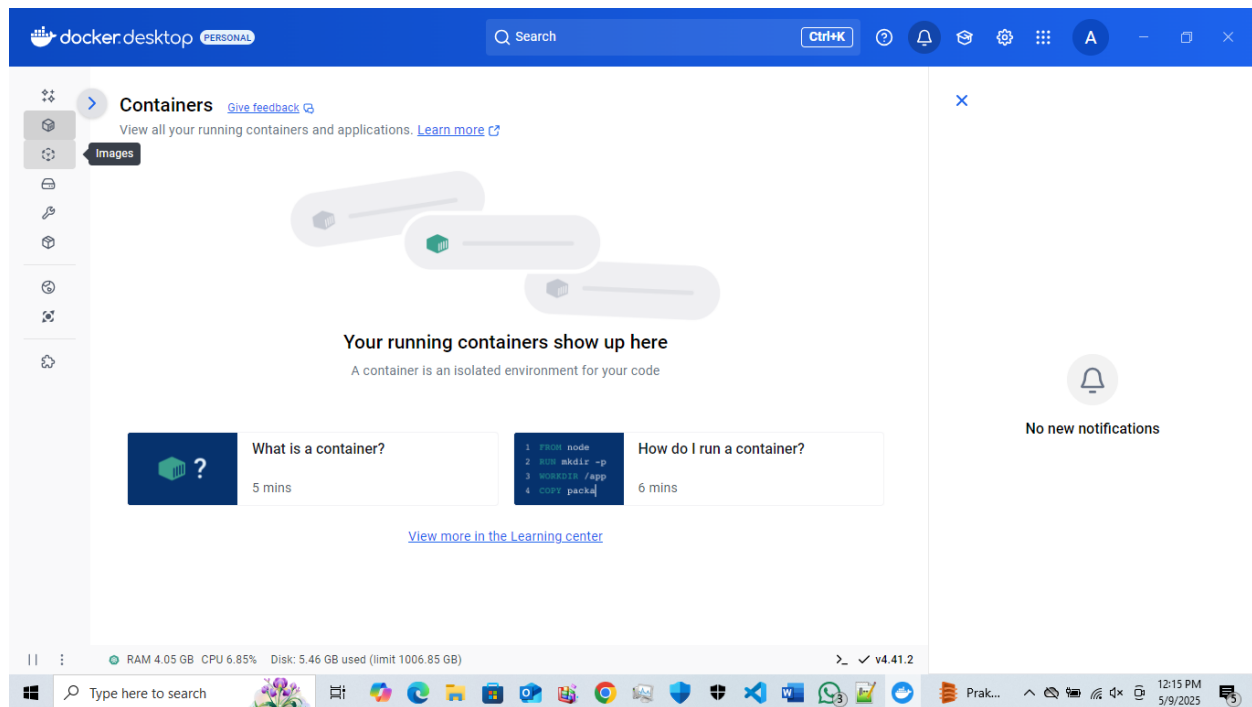
The line containing the `<image>` tag is highlighted in blue. The status bar at the bottom shows 'length: 16,169 lines: 421'.

Wait until the process is completed.

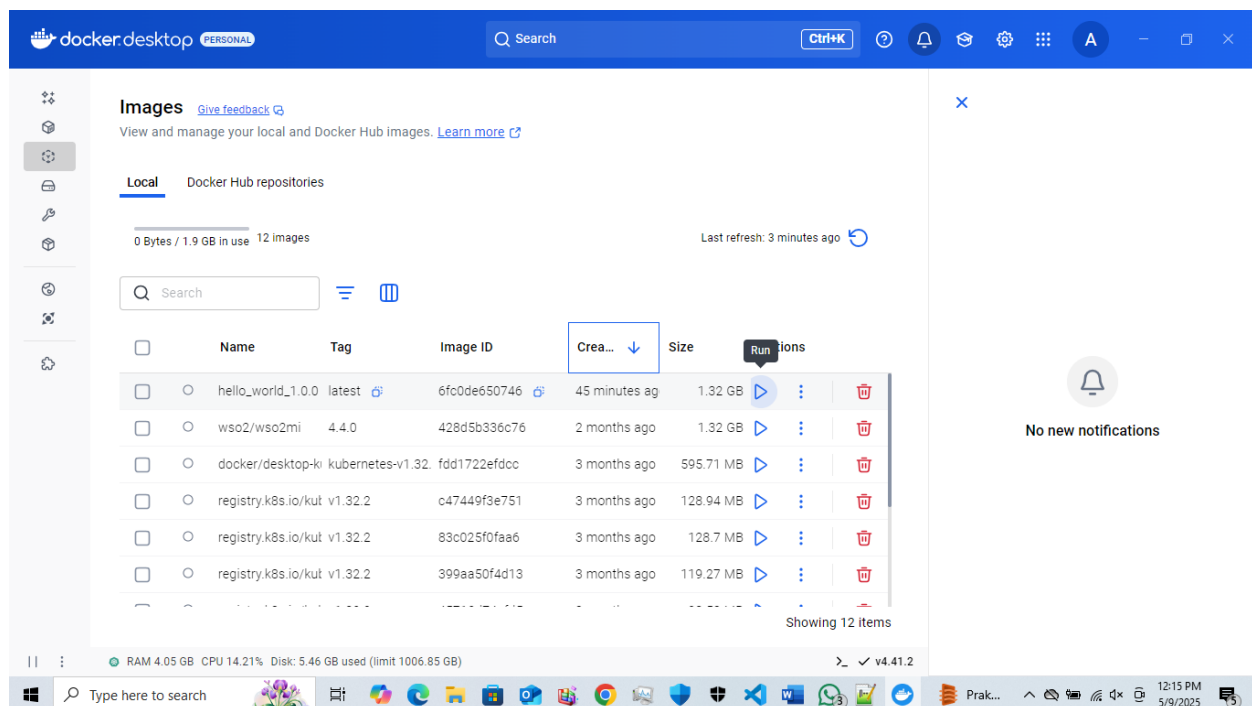


TESTING THE IMAGE BEING DEPLOYED

Click on image



Find the image

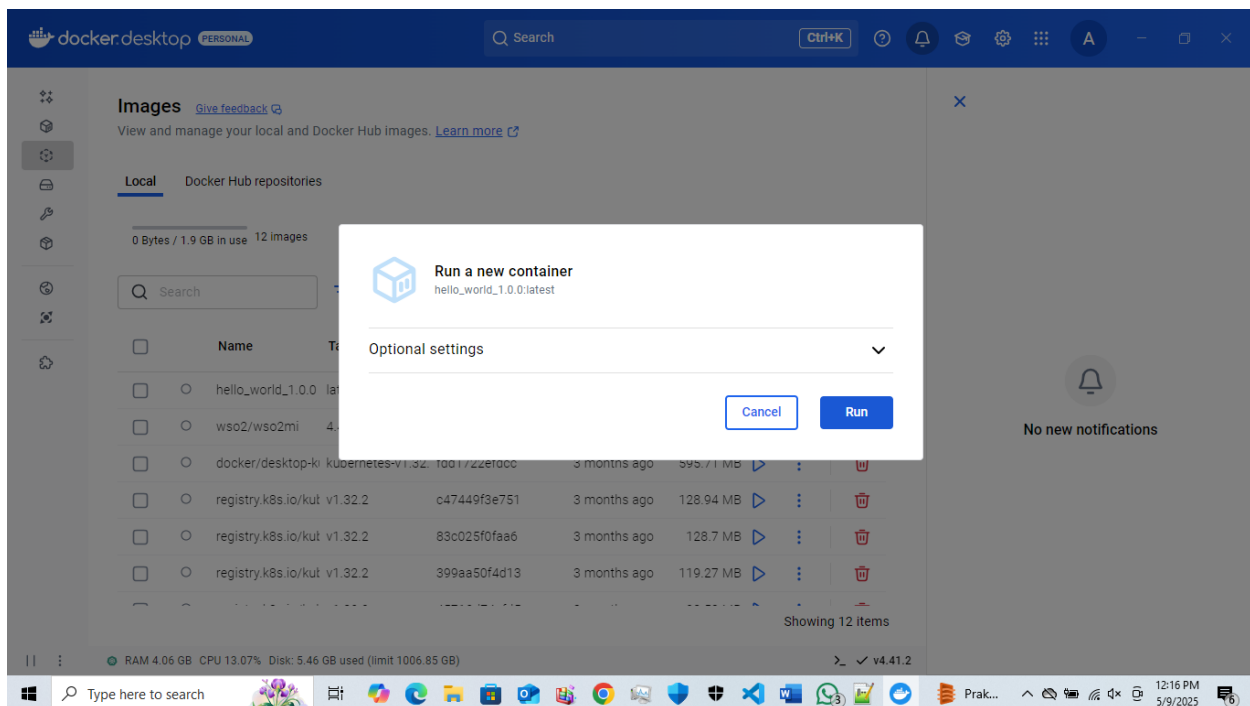
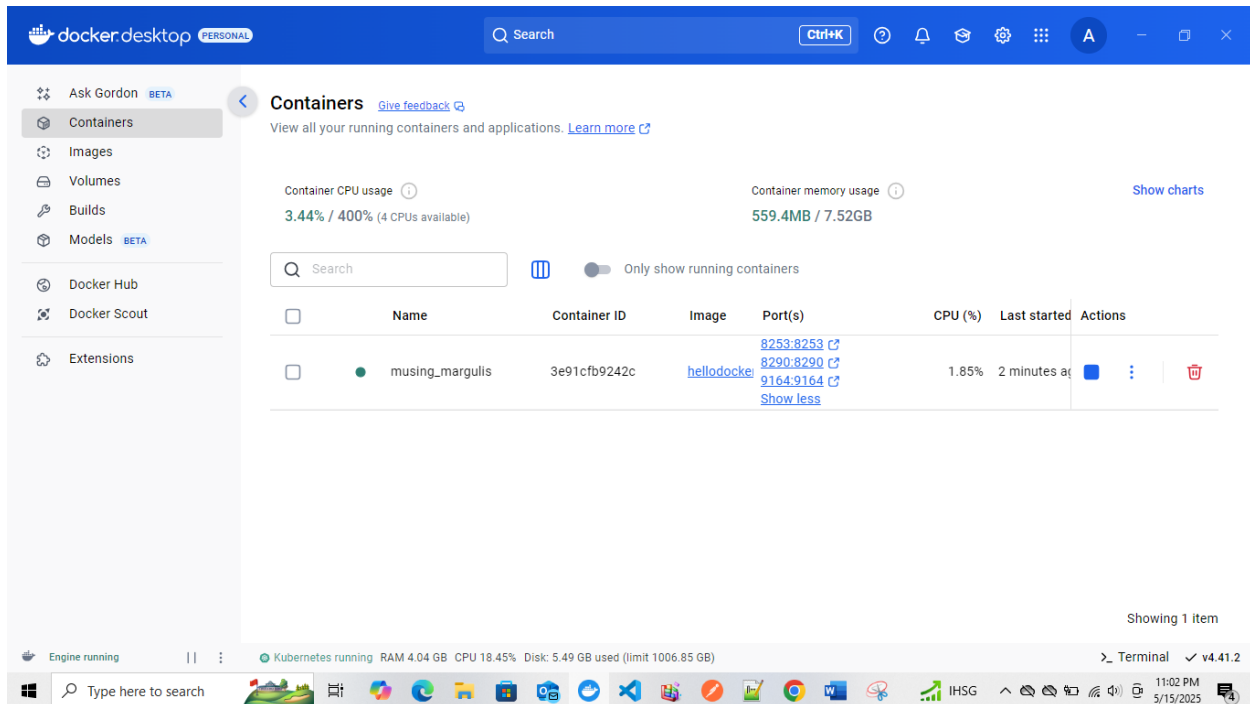


Open cmd and run the following command

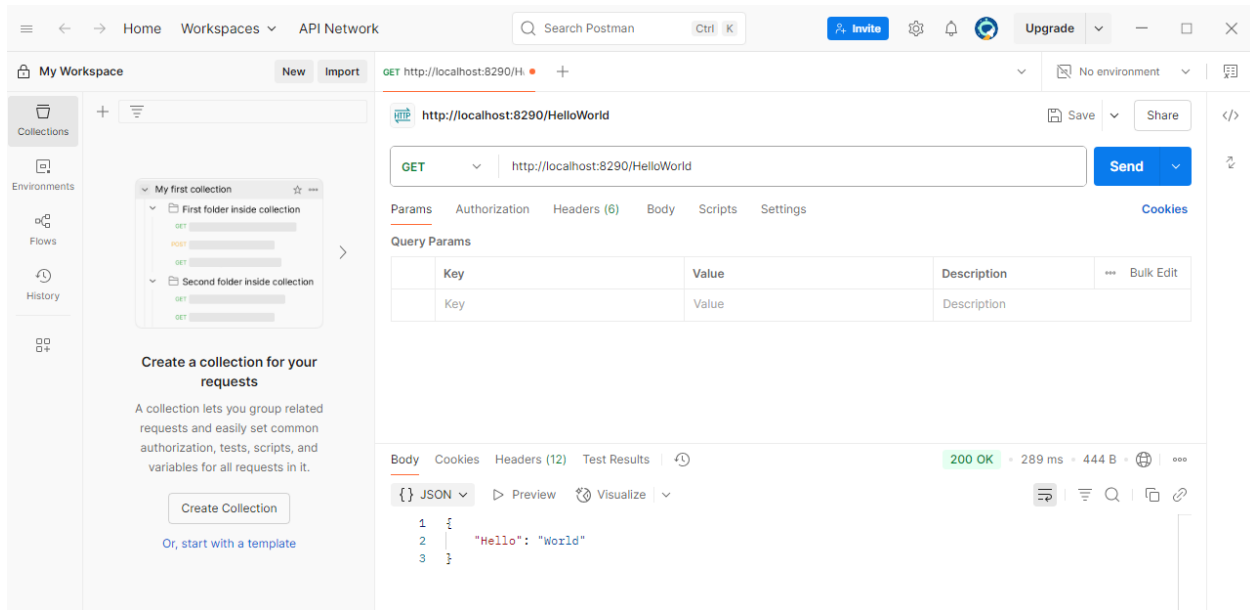
```
docker run -p 8290:8290 -p 8253:8253 -p 9164:9164 hellodocker:1.0.0
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\wso2mi\Projects\HelloDocker> docker run -p 8290:8290 -p 8253:8253 -p 9164:9164 hellodocker:1.0.0
[2025-05-15 16:01:21,038] INFO {ClassMediatorDeployer} - Deploying Class mediators from file : /home/wso2carbon/wso2mi-4.4.0/tmp/carbonapps/-1234/1747324881004HelloDocker_1.0.0.car/HelloDocker_class_mediators_1.0.0/HelloDocker-1.0.0.jar
```

Go to docker desktop to verify the container is running



Testing in postman/soap ui



Configuring the pods in Kubernetes so it can be accessed from the outside call.

☐		k8s_ags-integrator-container_ags-integrator-deploym	dcd8f1d2c79f	ariadana/ags_integrator		:			
☐		k8s_ags-integrator-container_ags-integrator-deploym	9181fc7a4cb6	ariadana/ags_integrator		:			
☐		k8s_ags-integrator-container_ags-integrator-deploym	47b20d96c9f0	ariadana/ags_integrator		:			

Edit the service yaml file .

```
apiVersion: v1
kind: Service
metadata:
  name: ags-integrator-service # A name for your service
spec:
  selector:
    app: ags-integrator # Selects pods with the 'app: my-app' label (matching the deployment template)
  ports:
    - protocol: TCP
      port: 8290 # The port the service will listen on
      targetPort: 8290 # The port the service will forward traffic to on the pods (matching containerPort)
  type: LoadBalancer # Or ClusterIP (internal) or NodePort (for local/testing)
```

Updated service yaml file

apiVersion: v1

kind: Service

metadata:

name: ags-integrator-service # A name for your service

spec:

selector:

app: ags-integrator # Selects pods with the 'app: my-app' label (matching the deployment template)

ports:

- protocol: TCP

port: 8290 # The port the service will listen on

targetPort: 8290 # The port the service will forward traffic to on the pods (matching containerPort)

type: LoadBalancer # Or ClusterIP (internal) or NodePort (for local/testing)

Apply the changes by command : `kubectl apply -f <yaml service name>`

Ex : `kubectl apply -f ags-integrator-service.yaml`

```
C:\Users\ariad\OneDrive\Documents\My Study\Self Study\Hands on Kubernetes>kubectl apply -f ags-integrator-service.yaml
service/ags-integrator-service configured
```

Check the port mapping by command : `kubectl get services`

```
C:\Users\ariad\OneDrive\Documents\My Study\Self Study\Hands on Kubernetes>kubectl get services
NAME                                TYPE                CLUSTER-IP      EXTERNAL-IP      PORT(S)          AGE
ags-integrator-service              LoadBalancer        10.96.118.35    localhost        8290:31742/TCP   2d22h
kubernetes                           ClusterIP             10.96.0.1       <none>           443/TCP          11d
```

Test by curl to the new port command : `curl -X GET <url>`

Ex : `curl -X GET http://localhost:8290/HelloWorld`

DEPLOYING MICRO INTEGRATOR CONTROL PLANE TO DOCKER

From CMD run the following command : `docker pull wso2/wso2-integration-control-plane`

```
Command Prompt - docker pull wso2/wso2-integration-control-plane
Microsoft Windows [Version 10.0.19045.5737]
(c) Microsoft Corporation. All rights reserved.

C:\Users\ariad>docker pull wso2/wso2-integration-control-plane
Using default tag: latest
latest: Pulling from wso2/wso2-integration-control-plane
72ca8b3248b8: Download complete
fdf9fe9233ad: Downloading [=>] 4.194MB/192.4MB
e55eabfb2fc4: Download complete
0d426c582892: Downloading [====>] 3.146MB/27.24MB
8bf0daebdca9: Download complete
3713021b0277: Downloading [====>] 3.146MB/29.53MB
0fd2e36ef243: Downloading [=====>] 3.146MB/22.78MB
4f4fb700ef54: Pull complete
```

Wait until the pull is completed

```
Command Prompt
Microsoft Windows [Version 10.0.19045.5737]
(c) Microsoft Corporation. All rights reserved.

C:\Users\ariad>docker pull wso2/wso2-integration-control-plane
Using default tag: latest
latest: Pulling from wso2/wso2-integration-control-plane
72ca8b3248b8: Pull complete
fdf9fe9233ad: Pull complete
e55eabfb2fc4: Pull complete
0d426c582892: Pull complete
8bf0daebdca9: Pull complete
3713021b0277: Pull complete
0fd2e36ef243: Pull complete
4f4fb700ef54: Pull complete
Digest: sha256:800cd4e2b430eed849a67633682536f55757b08658bba38fdd2f9b83416e6ddb
Status: Downloaded newer image for wso2/wso2-integration-control-plane:latest
docker.io/wso2/wso2-integration-control-plane:latest

C:\Users\ariad>
```

Run the following command in cmd

`$ docker run -p 9743:9743 --name integration-control-plane -d wso2/wso2-integration-control-plane:latest`

```
C:\Users\ariad>docker run -p 9743:9743 --name integration-control-plane -d wso2/wso2-integration-control-plane:latest
052f39bfa0bbcb23107c2ff7f34e6beda73fe3b12fae915b187d937c5f62a286

C:\Users\ariad>
```

Command to log the integrator control plane : `docker logs -f integration-control-plane`

```
Command Prompt - docker logs -f integration-control-plane
latest: Pulling from wso2/wso2-integration-control-plane
72ca8b3248b8: Pull complete
fd99fe9233ad: Pull complete
e55eabfb2fc4: Pull complete
0d426c582892: Pull complete
8bf0daebdca9: Pull complete
3713021b0277: Pull complete
0fd2e36ef243: Pull complete
4f4fb700ef54: Pull complete
Digest: sha256:800cd4e2b430eed849a67633682536f55757b08658bba38fdd2f9b83416e6ddb
Status: Downloaded newer image for wso2/wso2-integration-control-plane:latest
docker.io/wso2/wso2-integration-control-plane:latest

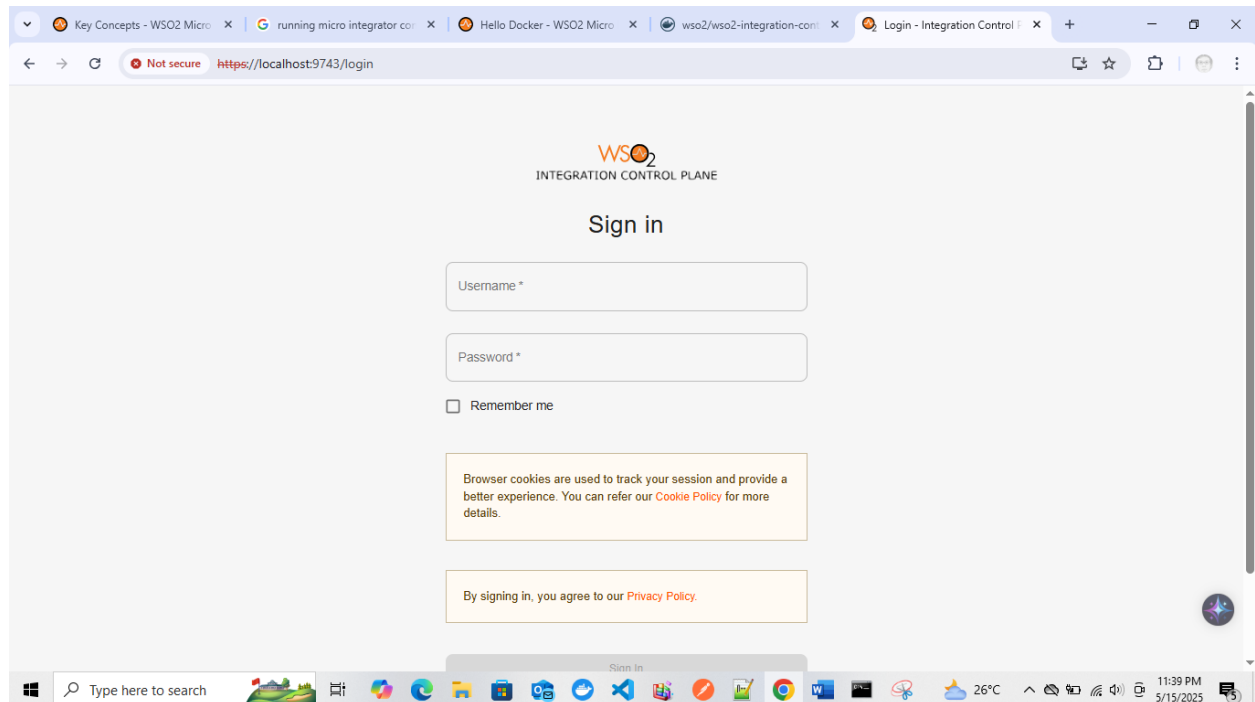
C:\Users\ariad>docker run -p 9743:9743 --name integration-control-plane -d wso2/wso2-integration-control-plane:latest
952f39bfa0bbcb23107c2ff7f34e6beda73fe3b12fae915b187d937c5f62a286

C:\Users\ariad>$ docker logs -f integration-control-plane
'$' is not recognized as an internal or external command,
operable program or batch file.

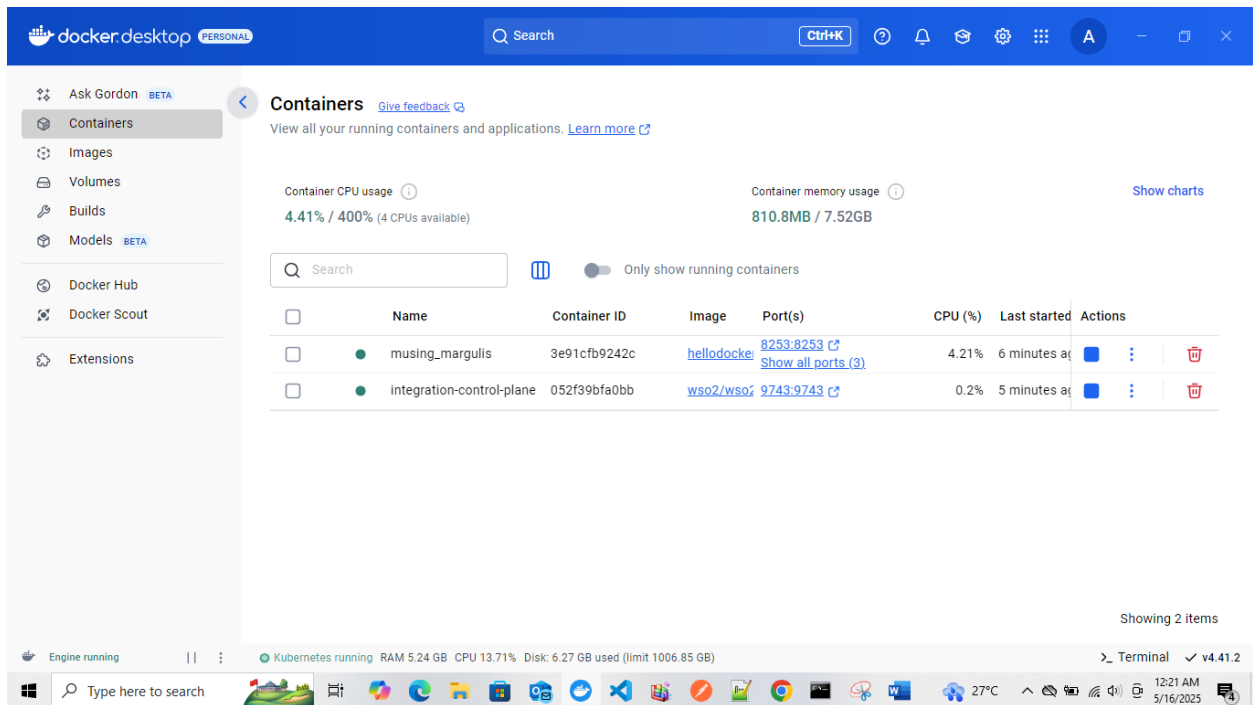
C:\Users\ariad>$ docker logs -f integration-control-plane
'$' is not recognized as an internal or external command,
operable program or batch file.

C:\Users\ariad>docker logs -f integration-control-plane
[2025-05-15 16:36:34,929] INFO {ConfigParser} - Initializing configurations with deployment configurations
[2025-05-15 16:36:38,873] INFO {DashboardServer} - WSO2 Integration Control Plane started.
[2025-05-15 16:36:38,874] INFO {DashboardServer} - Login to Integration Control Plane Dashboard : 'https://052f39bfa0bb
:9743/login'
```

Test the integrator control plane by entering url in browser : `https://localhost:9743/login`



CREATING NETWORK FOR THE CONTAINER



Goal : put container musing_margulis and integration-control-plane in the same network

Create the network : docker network create <network name>

Ex : docker network create localDockerNetwork

```
C:\Users\ariad>docker network create localDockerNetwork
7e24c60bd062d42542316d6708c447a837ffadd07743a7b584b5980edc5d25b6
```

Put the first container into the network : docker network connect <network name>
<container name>

Ex : docker network connect localDockerNetwork musing_margulis

Put the second container into the network : docker network connect localDockerNetwork
integration-control-plane

```
C:\Users\ariad>docker network connect localDockerNetwork musing_margulis
C:\Users\ariad>docker network connect localDockerNetwork integration-control-plane
```

Check the newly created network

docker network inspect <network name>

Ex : docker network inspect localDockerNetwork

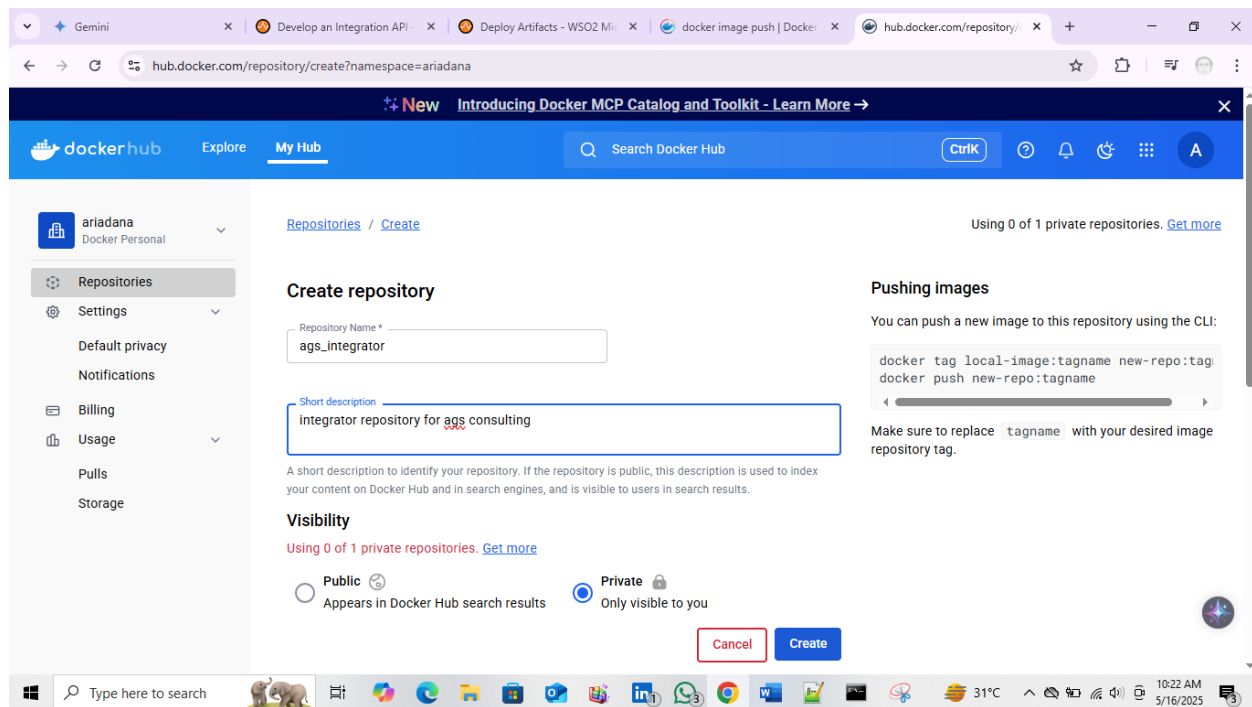
```
C:\Users\ariad>docker network inspect localDockerNetwork
[
  {
    "Name": "localDockerNetwork",
    "Id": "7e24c60bd062d42542316d6708c447a837ffadd07743a7b584b5980edc5d25b6",
    "Created": "2025-05-15T17:11:00.582594967Z",
    "Scope": "local",
    "Driver": "bridge",
    "EnableIPv4": true,
    "EnableIPv6": false,
    "IPAM": {
      "Driver": "default",
      "Options": {},
      "Config": [
        {
          "Subnet": "172.18.0.0/16",
          "Gateway": "172.18.0.1"
        }
      ]
    },
    "Internal": false,
    "Attachable": false,
    "Ingress": false,
    "ConfigFrom": {
      "Network": ""
    },
    "ConfigOnly": false,
    "Containers": {
```

Make sure both containers in the network

```
      "Containers": {
        "052f39bfa0bbcb23107c2ff7f34e6beda73fe3b12fae915b187d937c5f62a286": {
          "Name": "integration-control-plane",
          "EndpointID": "a9556e1516dbdec4945e3544aeb3a50812a64619f89357d808a3516b3213d825",
          "MacAddress": "52:68:2c:16:8a:18",
          "IPv4Address": "172.18.0.3/16",
          "IPv6Address": ""
        },
        "3e91cfb9242c6a6890f5df6b4c3ed8c7423aa97fed622797a0db5f28fb571fb2": {
          "Name": "musing_margulis",
          "EndpointID": "30cb2be2740c8438fb34f8bd7e3e2d4fddfdeaaf4c11db7e2823a327b4bcf4d1",
          "MacAddress": "46:75:a4:44:b7:e2",
          "IPv4Address": "172.18.0.2/16",
          "IPv6Address": ""
        }
      }
    }
  }
]
```

PUSH THE IMAGE INTO THE DOCKER HUB REPO

Create repository in docker hub, name the repo and click create.



Open cmd and write command : docker login

```
Command Prompt
Microsoft Windows [Version 10.0.19045.5737]
(c) Microsoft Corporation. All rights reserved.

C:\Users\ariad>docker login
Authenticating with existing credentials... [Username: ariadana]

Info → To login with a different account, run 'docker logout' followed by 'docker login'

Login did not succeed, error: error during connect: Post "http://%2F%2F.%2Fpipe%2FdockerDesktopLinuxEngine/v1.49/auth":
open //./pipe/dockerDesktopLinuxEngine: The system cannot find the file specified.

USING WEB-BASED LOGIN

Info → To sign in with credentials on the command line, use 'docker login -u <username>'
```

List all the images in the docker : docker images

```
Command Prompt
Your one-time device confirmation code is: LMFK-TRQF
Press ENTER to open your browser or submit your device code here: https://login.docker.com/activate

Waiting for authentication in the browser...
Login Succeeded

C:\Users\ariad>docker images
error during connect: Head "http://%2F%2F.%2Fpipe%2FdockerDesktopLinuxEngine/_ping": open //.pipe/dockerDesktopLinuxEngine: The system cannot find the file specified.

C:\Users\ariad>docker ls
docker: unknown command: docker ls

Run 'docker --help' for more information

C:\Users\ariad>docker images
error during connect: Head "http://%2F%2F.%2Fpipe%2FdockerDesktopLinuxEngine/_ping": open //.pipe/dockerDesktopLinuxEngine: The system cannot find the file specified.

C:\Users\ariad>docker images


| REPOSITORY                              | TAG                                                                         | IMAGE ID     | CREATED       | SIZE   |
|-----------------------------------------|-----------------------------------------------------------------------------|--------------|---------------|--------|
| hellodocker                             | 1.0.0                                                                       | 094652273a19 | 11 hours ago  | 1.33GB |
| hello_world_1.0.0                       | latest                                                                      | f6d0e101fd60 | 5 days ago    | 1.33GB |
| wso2/wso2mi                             | 4.4.0                                                                       | 428d5b336c76 | 2 months ago  | 1.33GB |
| docker/desktop-kubernetes               | kubernetes-v1.32.2-cni-v1.6.0-critools-v1.31.1-cri-dockerd-v0.3.16-1-debian | fd1722efdcc  | 2 months ago  | 596MB  |
| registry.k8s.io/kube-apiserver          | v1.32.2                                                                     | c47449f3e751 | 3 months ago  | 129MB  |
| registry.k8s.io/kube-controller-manager | v1.32.2                                                                     | 399aa50f4d13 | 3 months ago  | 119MB  |
| registry.k8s.io/kube-scheduler          | v1.32.2                                                                     | 45710d74cfd5 | 3 months ago  | 93.5MB |
| registry.k8s.io/kube-proxy              | v1.32.2                                                                     | 83c025f0faa6 | 3 months ago  | 129MB  |
| registry.k8s.io/etcd                    | 3.5.16-0                                                                    | c6a9d11cc5c0 | 8 months ago  | 211MB  |
| registry.k8s.io/coredns/coredns         | v1.11.3                                                                     | 9caabbf6238b | 9 months ago  | 85.1MB |
| wso2/wso2-integration-control-plane     | latest                                                                      | 800cd4e2b430 | 9 months ago  | 809MB  |
| registry.k8s.io/pause                   | 3.10                                                                        | ee6521f290b2 | 11 months ago | 1.06MB |
| docker/desktop-vpnkit-controller        | dc331cb22850be0cdd97c84a9cfecaf44a1afb6e                                    | 7ecf567ea070 | 2 years ago   | 47MB   |
| docker/desktop-storage-provisioner      | v2.0                                                                        | 115d77efe6e2 | 4 years ago   | 59.2MB |


C:\Users\ariad>
```

Tag docker image : `docker <image id> <docker hub username>/<repo name>:latest`

Ex : `docker tag 094652273a19 ariadana/ags_integrator:latest`

Confirm the tag has already been created by : `docker images`

```
C:\Users\ariad>docker images


| REPOSITORY                              | TAG                                                                         | IMAGE ID     | CREATED       | SIZE   |
|-----------------------------------------|-----------------------------------------------------------------------------|--------------|---------------|--------|
| ariadana/ags_integrator                 | latest                                                                      | 094652273a19 | 12 hours ago  | 1.33GB |
| hellodocker                             | 1.0.0                                                                       | 094652273a19 | 12 hours ago  | 1.33GB |
| hello_world_1.0.0                       | latest                                                                      | f6d0e101fd60 | 5 days ago    | 1.33GB |
| wso2/wso2mi                             | 4.4.0                                                                       | 428d5b336c76 | 2 months ago  | 1.33GB |
| docker/desktop-kubernetes               | kubernetes-v1.32.2-cni-v1.6.0-critools-v1.31.1-cri-dockerd-v0.3.16-1-debian | fd1722efdcc  | 2 months ago  | 596MB  |
| registry.k8s.io/kube-apiserver          | v1.32.2                                                                     | c47449f3e751 | 3 months ago  | 129MB  |
| registry.k8s.io/kube-controller-manager | v1.32.2                                                                     | 399aa50f4d13 | 3 months ago  | 119MB  |
| registry.k8s.io/kube-scheduler          | v1.32.2                                                                     | 45710d74cfd5 | 3 months ago  | 93.5MB |
| registry.k8s.io/kube-proxy              | v1.32.2                                                                     | 83c025f0faa6 | 3 months ago  | 129MB  |
| registry.k8s.io/etcd                    | 3.5.16-0                                                                    | c6a9d11cc5c0 | 8 months ago  | 211MB  |
| registry.k8s.io/coredns/coredns         | v1.11.3                                                                     | 9caabbf6238b | 9 months ago  | 85.1MB |
| wso2/wso2-integration-control-plane     | latest                                                                      | 800cd4e2b430 | 9 months ago  | 809MB  |
| registry.k8s.io/pause                   | 3.10                                                                        | ee6521f290b2 | 11 months ago | 1.06MB |
| docker/desktop-vpnkit-controller        | dc331cb22850be0cdd97c84a9cfecaf44a1afb6e                                    | 7ecf567ea070 | 2 years ago   | 47MB   |
| docker/desktop-storage-provisioner      | v2.0                                                                        | 115d77efe6e2 | 4 years ago   | 59.2MB |


```

Push the image to docker desktop : `docker push <docker hub username>/<repo name>:latest`

Ex : `docker push ariadana/ags_integrator:latest`

Confirm the image in the docker hub

ariadana/ags_integrator

Last pushed 2 minutes ago • Repository size: 466.2 MB

integrator repository for ags consulting 

Add a category 

General

Tags

Image Management **BETA**

Collaborators

Webhooks

Set

Tags

 DOCKER SCOUT INACTIVE
[Activate](#)

This repository contains 0 tag(s).

Tag	OS	Type	Pulled	Pushed
 latest		Image	less than 1 day	2 minutes

[See all](#)

Create Kubernetes deployment file (yaml). Filename <appname>-deployment.yaml

Ex : ags-integrator-deployment.yaml

apiVersion: apps/v1

kind: Deployment

metadata:

name: ags-integrator-deployment # A name for your deployment

spec:

replicas: 3 # How many instances (Pods) you want to run

selector:

matchLabels:

app: ags-integrator # Selects pods with this label

```
template: # Defines the pods created by this deployment

metadata:

  labels:

    app: ags-integrator # Label for the pods

spec:

  containers:

    - name: ags-integrator-container # Name for the container within the pod

      image: ariadana/ags_integrator:latest # Replace with your image tag

    ports:

      - containerPort: 8080 # The port your application listens on inside the container

      # Add resource requests/limits, environment variables, volume mounts here
```

Create service yaml : <appname>_service.yaml

Ex : ags-integrator-service.yaml

```
apiVersion: v1

kind: Service

metadata:

  name: ags-integrator-service # A name for your service

spec:

  selector:

    app: ags-integrator # Selects pods with the 'app: my-app' label (matching the deployment
template)

  ports:

    - protocol: TCP

    port: 80 # The port the service will listen on
```

targetPort: 8080 # The port the service will forward traffic to on the pods (matching containerPort)

type: LoadBalancer # Or ClusterIP (internal) or NodePort (for local/testing)

Open cmd and go to the deployment and service yaml file located.

Execute deployment and service yaml to Kubernetes.

Command :

kubectl apply -f ags-integrator-deployment.yaml

kubectl apply -f ags-integrator-service.yaml

```
C:\Users\ariad\OneDrive\Documents\My Study\Self Study\Hands on Kubernetes>kubectl apply -f ags-integrator-deployment.yaml
deployment.apps/ags-integrator-deployment created

C:\Users\ariad\OneDrive\Documents\My Study\Self Study\Hands on Kubernetes>kubectl apply -f ags-integrator-service.yaml
service/ags-integrator-service created
```

Checking the deployment command : kubectl get deployments

```
C:\Users\ariad\OneDrive\Documents\My Study\Self Study\Hands on Kubernetes>kubectl get deployments
NAME                READY   UP-TO-DATE   AVAILABLE   AGE
ags-integrator-deployment  0/3     3            0           3m55s
```

Checking the pods command : kubectl get pods

```
C:\Users\ariad\OneDrive\Documents\My Study\Self Study\Hands on Kubernetes>kubectl get pods
NAME                                READY   STATUS             RESTARTS   AGE
ags-integrator-deployment-7cc885dc8-41sfq  0/1     ImagePullBackOff    0          7m39s
ags-integrator-deployment-7cc885dc8-7d9m5  0/1     ImagePullBackOff    0          7m39s
ags-integrator-deployment-7cc885dc8-gs7vn  0/1     ImagePullBackOff    0          7m39s
```

Checking the Kubernetes service : kubectl get services

```
C:\Users\ariad\OneDrive\Documents\My Study\Self Study\Hands on Kubernetes>kubectl get services
NAME                TYPE          CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
ags-integrator-service  LoadBalancer  10.96.118.35  localhost     80:31742/TCP     9m40s
kubernetes           ClusterIP      10.96.0.1     <none>        443/TCP          8d
```

If the Kubernetes has error regarding pull image, try to setup docker login in your docker machine (creating image pull secrets).

Command : create secret docker-registry my-dockerhub-secret --docker-server=docker.io -
-docker-username=<docker hub username> --docker-password=<docker hub password> -
-docker-email=<docker hub email address>

Editing the deployment yaml file (ags-integrator-deployment.yaml)

Add the following line :

```
imagePullSecrets: # <-- Add this section - name: my-dockerhub-secret # <-- Name of the
secret you created
```

```
  - name: my-dockerhub-secret # <-- Name of the secret you createdCompleted
deployment yaml file
```

```
apiVersion: apps/v1
```

```
kind: Deployment
```

```
metadata:
```

```
  name: ags-integrator-deployment # A name for your deployment
```

```
spec:
```

```
  replicas: 3 # How many instances (Pods) you want to run
```

```
  selector:
```

```
    matchLabels:
```

```
      app: ags-integrator # Selects pods with this label
```

```
  template: # Defines the pods created by this deployment
```

```
    metadata:
```

```
      labels:
```

```
        app: ags-integrator # Label for the pods
```

```
    spec:
```

```
      containers:
```

```
        - name: ags-integrator-container # Name for the container within the pod
```

```
          image: ariadana/ags_integrator:latest # Replace with your image tag
```

```
          ports:
```

```
            - containerPort: 8080 # The port your application listens on inside the container
```

```
          # Add resource requests/limits, environment variables, volume mounts here
```

imagePullSecrets: # <-- Add this section - name: my-dockerhub-secret # <-- Name of the secret you created

- name: my-dockerhub-secret # <-- Name of the secret you created

Update the yaml deployment file command : `kubectl apply -f your-deployment.yaml`

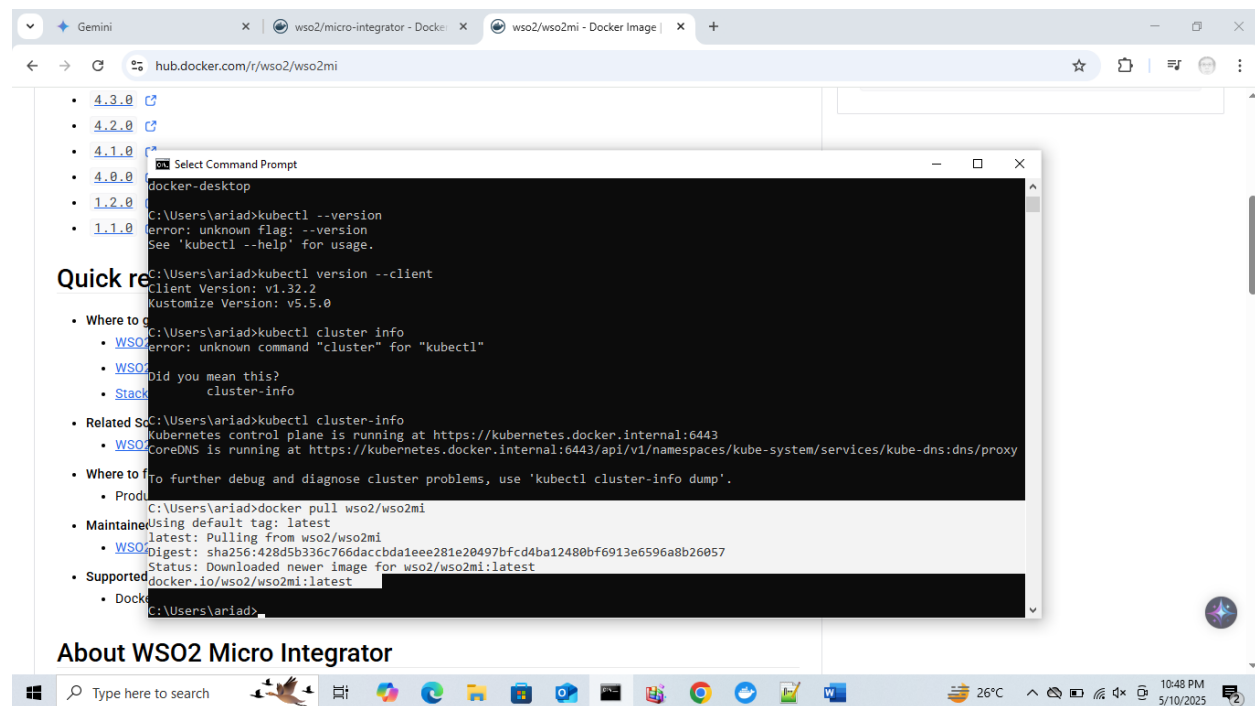
Ex : `kubectl apply -f ags-integrator-deployment.yaml`

```
C:\Users\ariad\OneDrive\Documents\My Study\Self Study\Hands on Kubernetes>kubectl apply -f ags-integrator-deployment.yaml
deployment.apps/ags-integrator-deployment configured
```

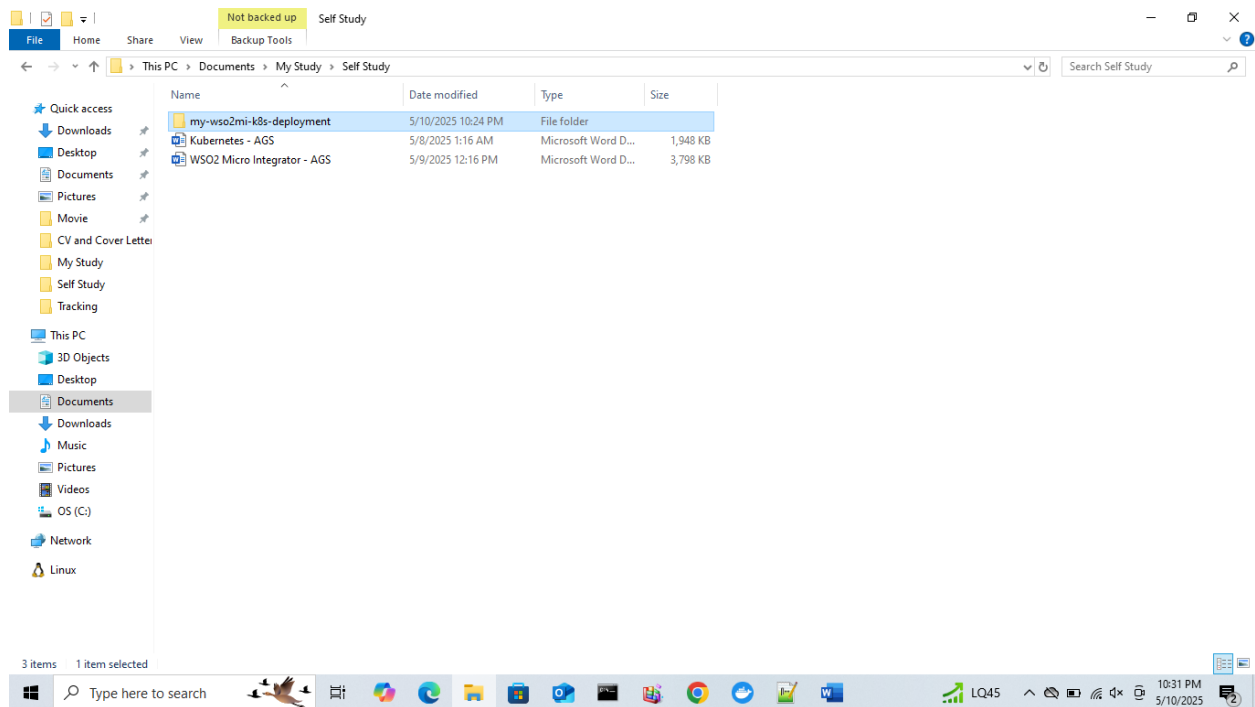
DEPLOY THE IMAGE TO KUBERNETES (WORK IN PROGRESS)

Get latest image for docker wso2 micro integrator

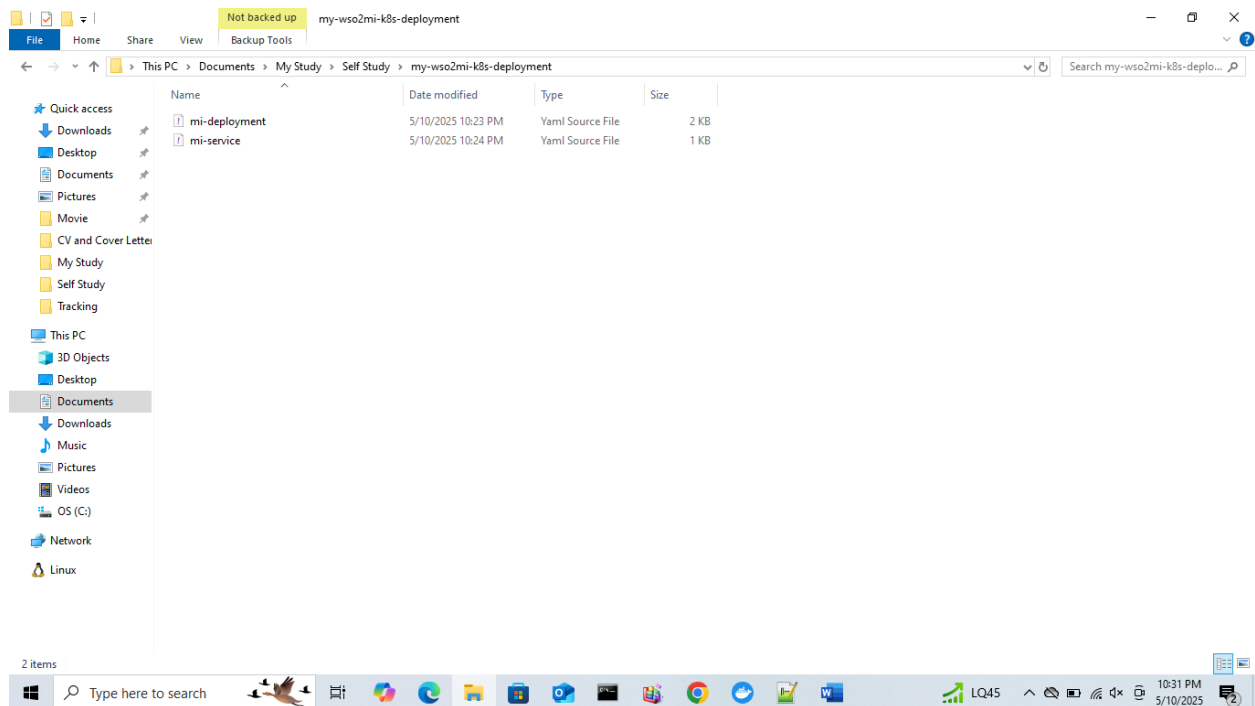
Run cmd and execute : `docker pull wso2/wso2mi`



Create directory for the deployment :



Inside the directory create yaml files :



mi-deployment.yaml

apiVersion: apps/v1

kind: Deployment

metadata:

name: wso2mi-deployment

labels:

app: wso2mi

spec:

replicas: 1 # Number of MI instances you want to run

selector:

matchLabels:

app: wso2mi

template:

metadata:

labels:

app: wso2mi

spec:

containers:

- name: wso2mi-container

image: wso2/wso2mi:latest # Or a specific version like wso2/wso2mi:4.2.0

ports:

Default ports for WSO2 Micro Integrator (MI 4.x and later)

- containerPort: 8290 # HTTP Pass-through transport

name: http-pt

- containerPort: 8253 # HTTPS Pass-through transport

name: https-pt

- containerPort: 9164 # Management API HTTP (internal)

name: mgmt-http

- containerPort: 9154 # Management API HTTPS (internal)

```
    name: mgmt-https

# --- Liveness and Readiness Probes (Recommended for robust deployments) ---

# livenessProbe:

#   httpGet:

#     path: /healthz # Assuming you have a health check endpoint or use one MI provides

#     port: 8290 # Or management port

#   initialDelaySeconds: 120 # Time to wait before first probe

#   periodSeconds: 10

# readinessProbe:

#   httpGet:

#     path: /healthz

#     port: 8290

#   initialDelaySeconds: 30

#   periodSeconds: 10

#   failureThreshold: 5
```

mi-service.yaml

apiVersion: v1

kind: Service

metadata:

name: wso2mi-service

spec:

selector:

app: wso2mi # Must match the 'app' label in your Deployment's template

ports:

- name: service-http

protocol: TCP

port: 8290 # Port the service will be available on within the cluster / externally

targetPort: http-pt # Name of the container port from the Deployment (or port number 8290)

- name: service-https

protocol: TCP

port: 8253

targetPort: https-pt # Or port number 8253

- name: service-mgmt-http # Exposing the Management API

protocol: TCP

port: 9164 # Exposing Management API on this port externally/within cluster

targetPort: mgmt-http # Name of the management HTTP port from deployment (or 9164)

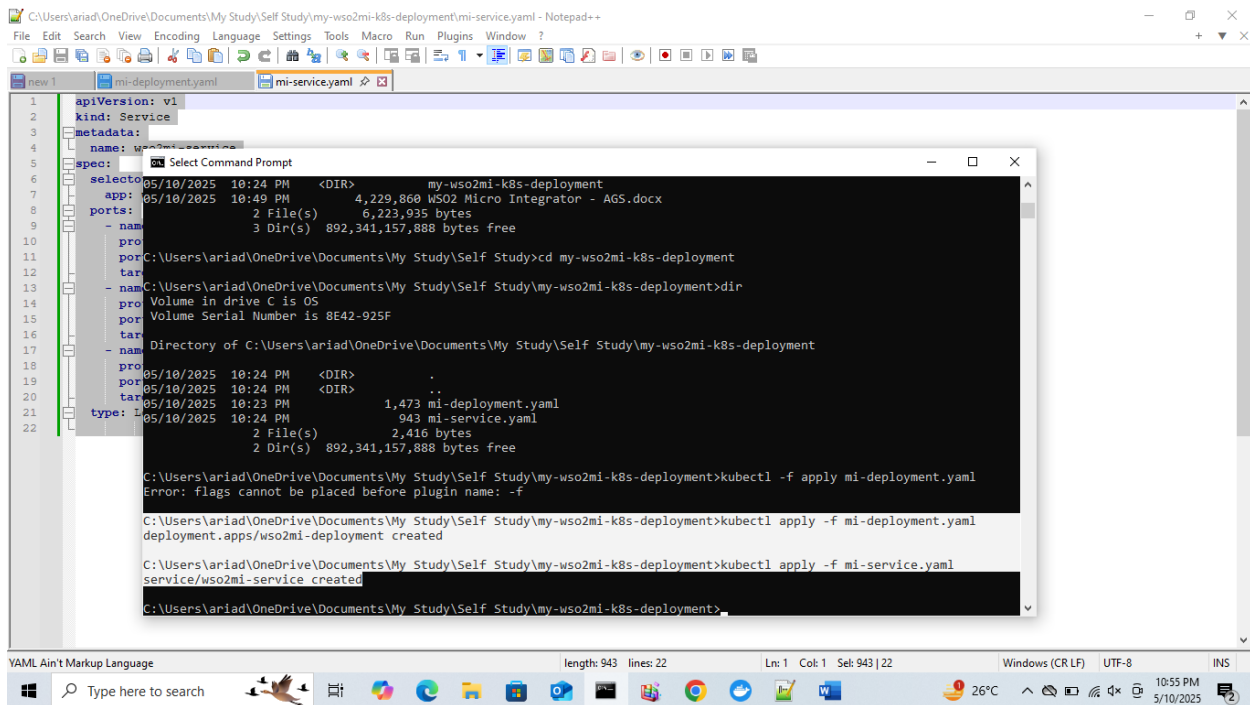
type: LoadBalancer # This type is convenient for Docker Desktop.

It makes the service accessible via localhost:<port> on your machine.

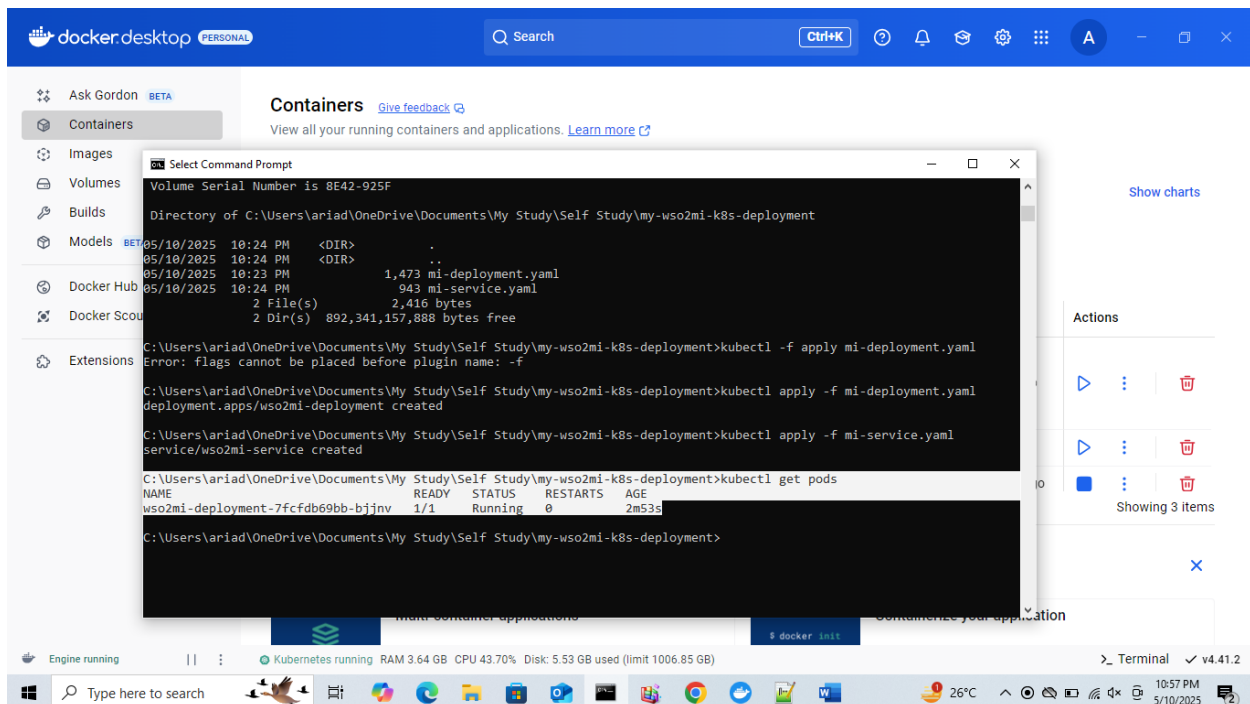
Execute the kubectl for deployment and service file in Kubernetes

kubectl apply -f mi-deployment.yaml

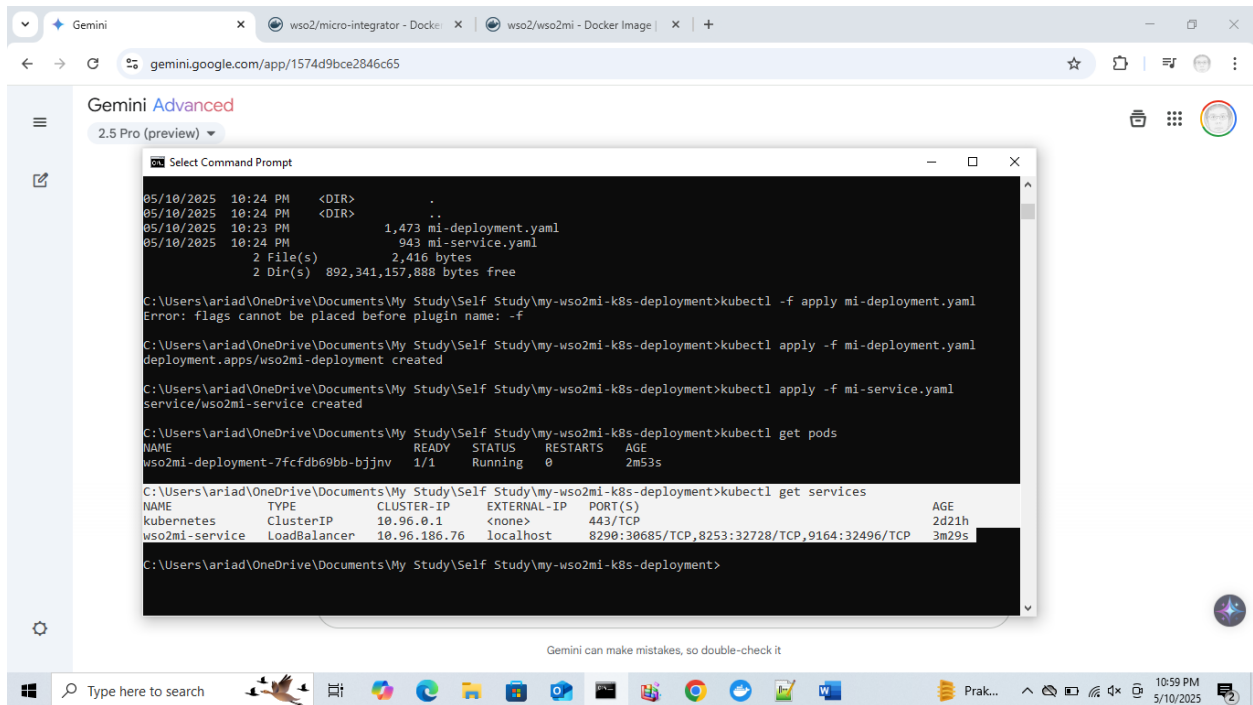
kubectl apply -f mi-service.yaml



Check the status : kubectl get pods



kubectl get services



Configuring run and debug for visual studio code.

Configure the debugger.

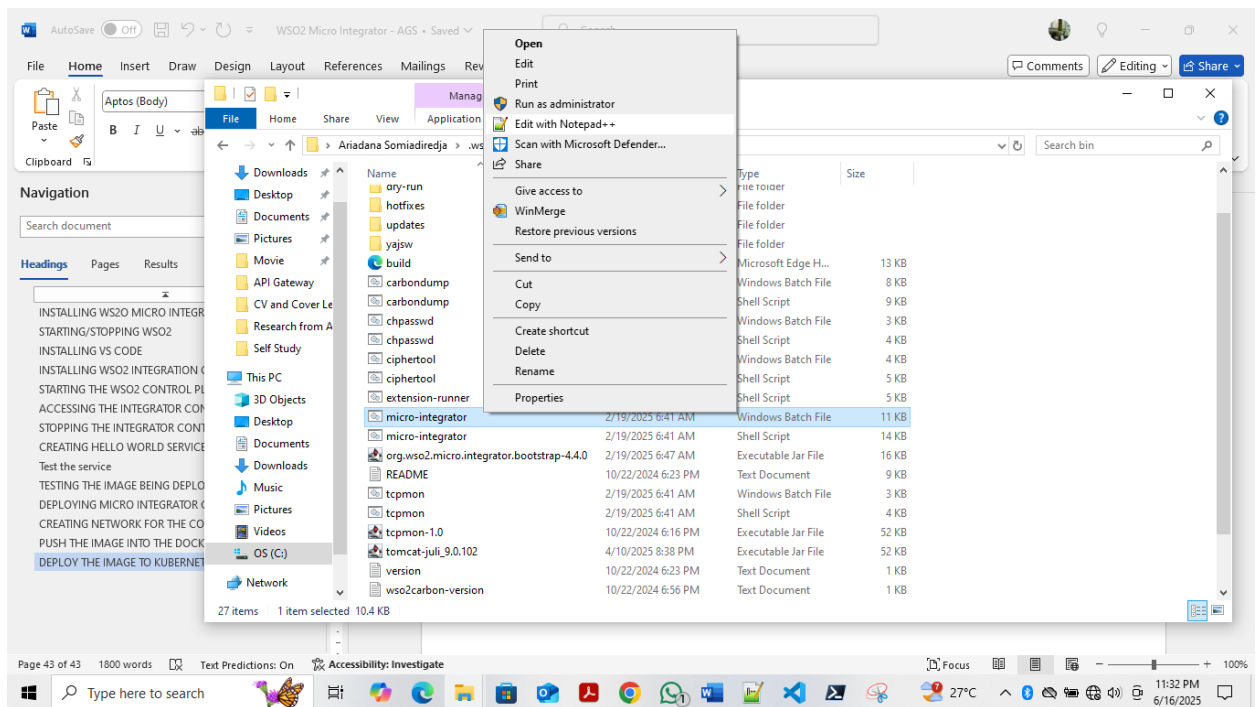
1. Go to the MI HOME directory.
cd \$env:MI_HOME, enter the bin directory.

```

C:\Users\ariad\.wso2-mi\micro-integrator\wso2mi-4.4.0
PS C:\Users\ariad> cd $env:MI_HOME
PS C:\Users\ariad\.wso2-mi\micro-integrator\wso2mi-4.4.0> cd bin

```

2. Edit the micro-integrator executable file.



Change the debugger port into 5005.

Replece the java opts line with this line

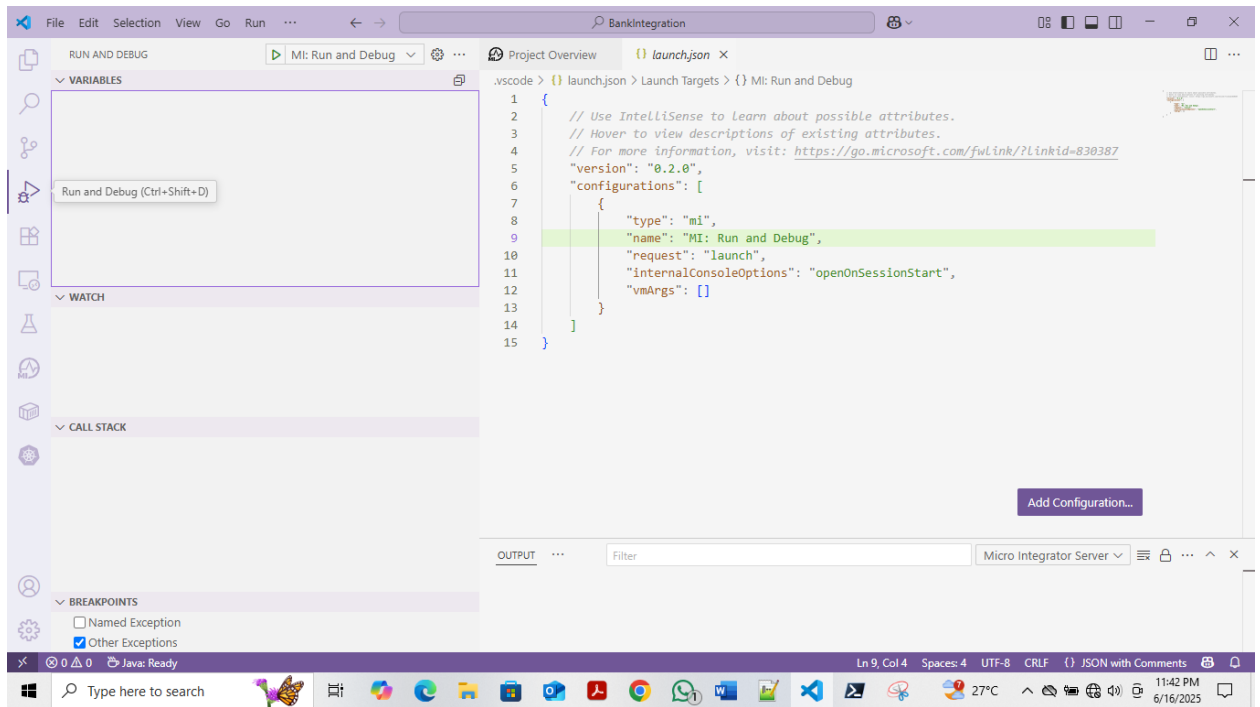
@REM Add this line near the top of the script

```
set JAVA_OPTS=-Xms256m -Xmx1024m -XX:+HeapDumpOnOutOfMemoryError -
XX:HeapDumpPath="%CARBON_HOME%\repository\logs\heap-dump.hprof" -
Djava.awt.headless=true -Djava.net.preferIPv4Stack=true -Xdebug -Xnoagent -
Djava.compiler=NONE -
Xrunjdwp:transport=dt_socket,server=y,suspend=n,address=5005
```

```
152 @REM Add this line near the top of the script
153 set JAVA_OPTS=-Xms256m -Xmx1024m -XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath="%CARBON_HOME%\repository\logs\heap-dump.hprof" -Djava.awt.headless=true -Djava
154 echo Please start the remote debugging client to continue...
155 goto findJdk
```

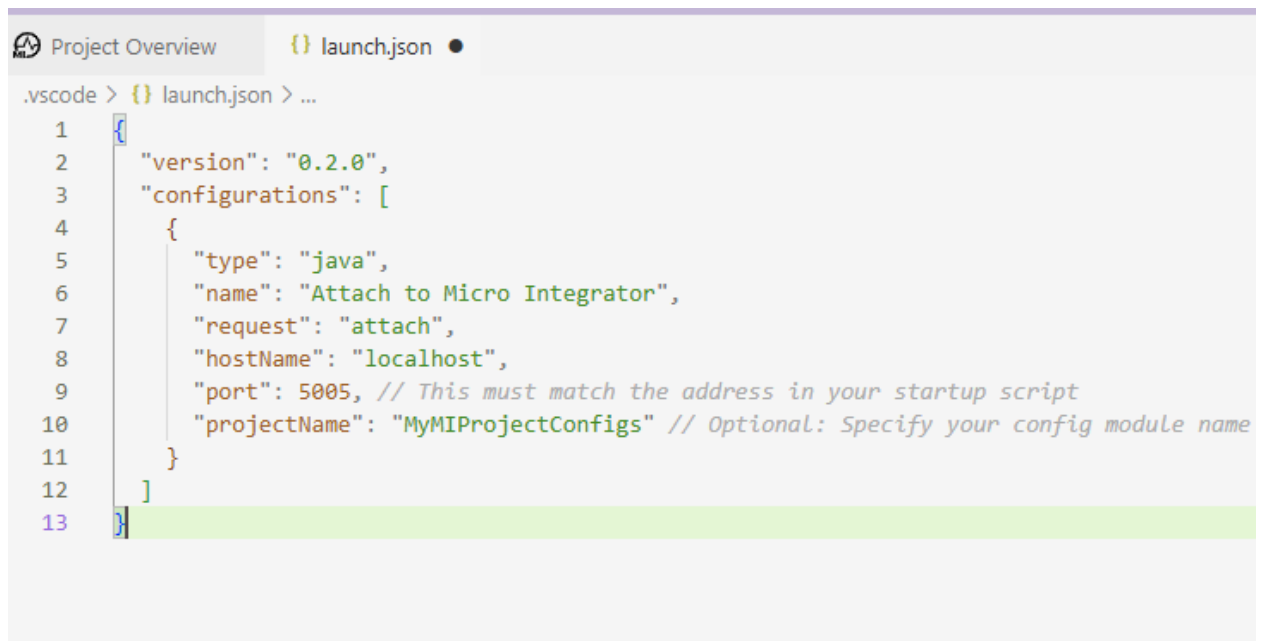
Create launch.json file in vs studi code.

Click the run and debug icon in the vs studio code.



Replace the launch.json file with the following :

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "type": "java",
      "name": "Attach to Micro Integrator",
      "request": "attach",
      "hostname": "localhost",
      "port": 5005, // This must match the address in your startup script
      "projectName": "MyMIPProjectConfigs" // Optional: Specify your config module
    }
  ]
}
```



The screenshot shows the Visual Studio Code editor interface. At the top, there is a tab labeled 'Project Overview' and another tab labeled 'launch.json'. The 'launch.json' tab is active, showing a JSON configuration file. The file content is as follows:

```
.vscode > {} launch.json > ...
1  {
2    "version": "0.2.0",
3    "configurations": [
4      {
5        "type": "java",
6        "name": "Attach to Micro Integrator",
7        "request": "attach",
8        "hostName": "localhost",
9        "port": 5005, // This must match the address in your startup script
10       "projectName": "MyMIPProjectConfigs" // Optional: Specify your config module name
11     }
12   ]
13 }
```

Save the json file.

Execute the command using debug port.

.\micro-integrator debug 5005